



同濟大學
TONGJI UNIVERSITY

计算机系统结构课程实验 总结报告

实验题目：简单的流水线 CPU 设计与性能分析

学 号： 2154312

姓 名： 郑博远

指导教师： 陆有军老师

日期： 2023.11.20

一、实验环境部署与硬件配置说明

- 系统环境：Windows 11
- 开发环境：Vivado 2016.2
- 硬件环境：Xilinx NEXYS4 DDR

二、实验的总体结构

本次实验的包含了对五段静态流水线的实现。该流水线总体结构分为五个阶段，分别是 IF（指令获取）、ID（指令译码）、EX（执行）、MEM（访存）、WB（写回）。该流水线的目标是实现对 31 条 MIPS 指令的五段流水线处理。

1. IF（指令获取）阶段：

- 负责从指令存储器中获取指令。
- 控制逻辑用于管理指令的流入，并将其传递到下一个阶段。

2. ID（指令译码）阶段：

- 负责对获取的指令进行译码，确定指令的类型和操作数。
- 识别操作数的寄存器地址并将其发送到执行阶段。
- 控制逻辑处理分支、跳转等控制指令。

3. EX（执行）阶段：

- 执行 ALU（算术逻辑单元）操作，进行算术和逻辑运算。
- 处理分支和跳转条件，计算分支目标地址。

4. MEM（访存）阶段：

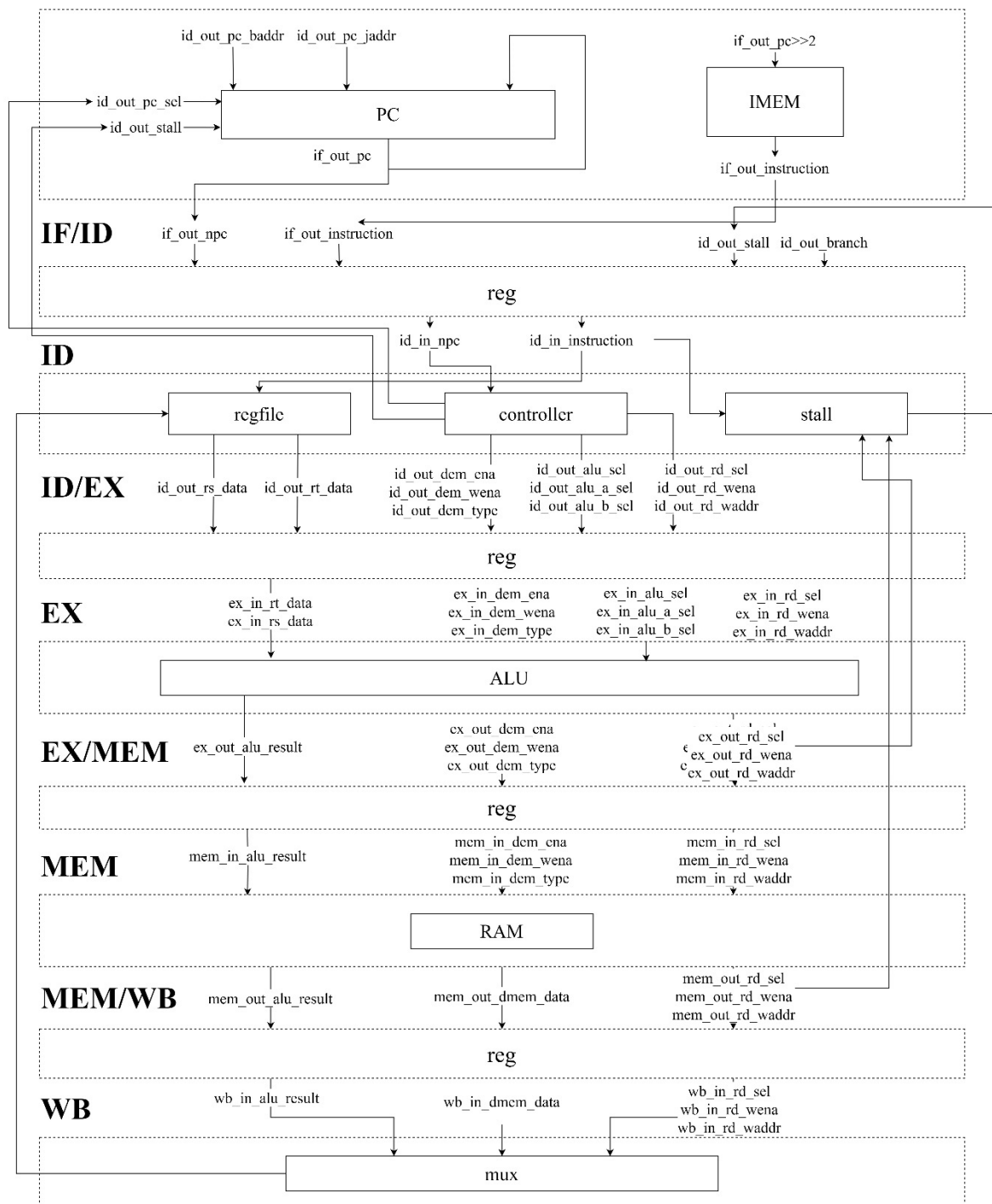
- 处理访存指令，包括加载和存储。
- 访问数据存储器，读取或写入数据。

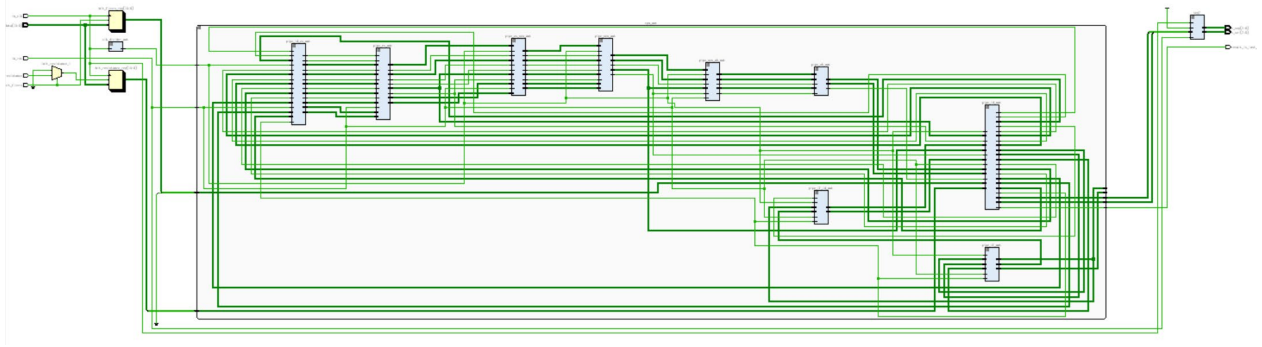
5. WB（写回）阶段：

- 将计算结果写回寄存器文件，更新寄存器的值。

上述的每个阶段都通过流水线寄存器连接,使得每个阶段的输出可以传递到下一个阶段,形成完整的流水线。这种结构允许在同一时钟周期内处理多个指令的不同阶段,提高了整体性能。通过实现这样的流水线结构,可以更有效地执行 MIPS 指令集中的各种操作。流水线整体架构图(略有省略)如下图所示:

IF





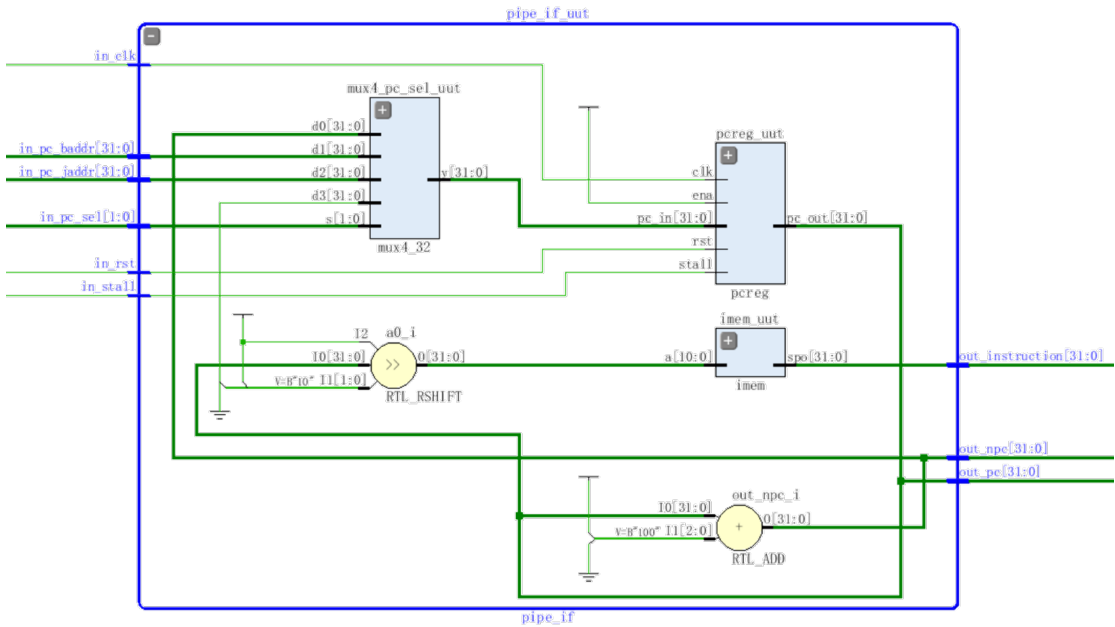
控制信号部分在上学期计算机组成原理课程的单周期 31 条 MIPS 指令集 CPU 基础上略作修改，详见下表：

	IM_R	DM_R	DM_W	ALUC	MUX_PC	MUX_A	MUX_B	MUX_Rd	Reg_W
add	1	0	0	0000	00	00	00	00	1
addu	1	0	0	0001	00	00	00	00	1
sub	1	0	0	0010	00	00	00	00	1
subu	1	0	0	0011	00	00	00	00	1
and	1	0	0	0100	00	00	00	00	1
or	1	0	0	0101	00	00	00	00	1
xor	1	0	0	0110	00	00	00	00	1
nor	1	0	0	0111	00	00	00	00	1
slt	1	0	0	1000	00	00	00	00	1
sltu	1	0	0	1001	00	00	00	00	1
sll	1	0	0	1010	00	10	00	00	1
srl	1	0	0	1011	00	10	00	00	1
sra	1	0	0	1100	00	10	00	00	1
sllv	1	0	0	1010	00	00	00	00	1
srlv	1	0	0	1011	00	00	00	00	1
srav	1	0	0	1100	00	00	00	00	1
jr	1	0	0		01				0
addi	1	0	0	0000	00	00	01	00	1
addiu	1	0	0	0001	00	00	01	00	1
andi	1	0	0	0100	00	00	01	00	1
ori	1	0	0	0101	00	00	01	00	1
xori	1	0	0	0110	00	00	01	00	1
lw	1	1	0	0000	00	00	01	01	1
sw	1	0	1	0000	00	00	01		0

beq	1	0	0	0011	eq?00:11	00	00		0
bne	1	0	0	0011	neq?00:11	00	00		0
slti	1	0	0	1000	00	00	01	00	1
sltiu	1	0	0	1001	00	00	01	00	1
lui	1	0	0	1101	00	00	01	00	1
j	1	0	0		10				0
jal	1	0	0	0000	10	01	10	00	1

三、总体架构部件的解释说明

1. IF（指令获取）阶段



```
// 指令内存访问模块
module pipe_if(
    input          in_clk,          // 时钟输入
    input          in_rst,          // 复位输入
    input          in_stall,        // 流水线暂停信号
    input  [31:0]  in_pc_jaddr,     // 跳转地址输入
    input  [31:0]  in_pc_baddr,     // 分支地址输入
    input  [1:0]   in_pc_sel,       // PC 选择信号，用于判断跳转或分支
    output [31:0]  out_pc,          // 输出 PC
    output [31:0]  out_npc,        // 输出下一个 PC

```

```

output [31:0] out_instruction // 输出指令
);

```

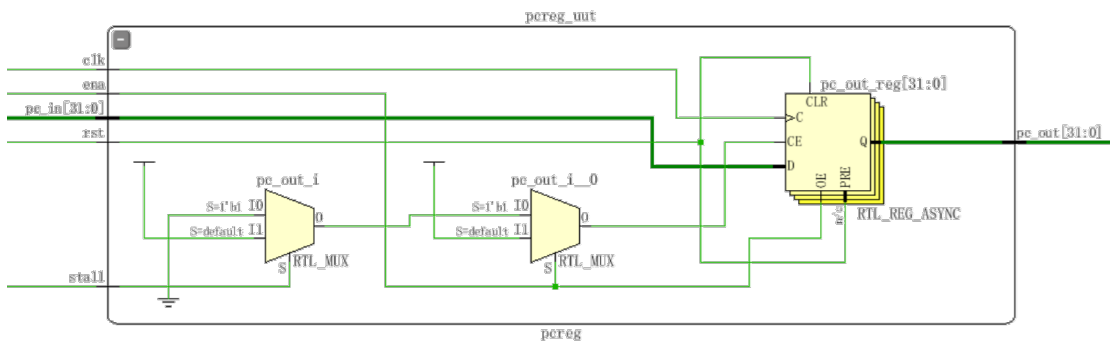
pipe_if 模块是流水线中的指令内存阶段的实现。该模块负责处理当前指令的地址、下一条指令的地址以及当前指令的内容。整体结构分为三个主要部分：

首先，实例化 pcreg 模块，该模块负责维护和更新当前指令的地址。其输入包括时钟信号 in_clk、复位信号 in_rst、暂停信号 in_stall、下一条指令的地址 next_pc，以及输出当前指令的地址 out_pc。

其次，使用了 mux4_32 模块，该模块是一个 4 选 1 的 32 位多路选择器，根据 in_pc_sel 选择不同的地址输入，输出到 next_pc 中，作为下一条指令的地址。其中，out_npc 通过简单的加法操作得到，表示当前指令的地址加上 4。

最后，通过 imem 模块读取指令内存，根据当前指令的地址 out_pc 获取相应的指令内容，将其输出为 out_instruction。

① pc 寄存器模块

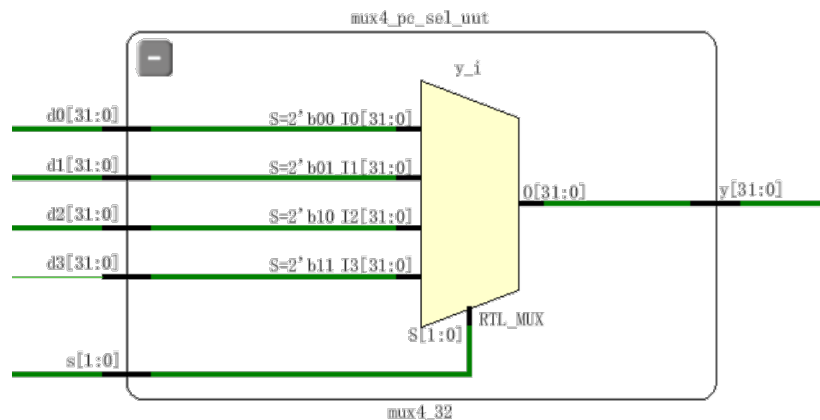


```

// 指令内存流水线寄存器模块
module pcreg(
    input          clk,          // 时钟信号
    input          ena,          // 使能信号
    input          rst,          // 复位信号
    input          stall,        // 暂停信号
    input [31:0]   pc_in,        // 输入地址
    output reg [31:0] pc_out     // 输出地址
);

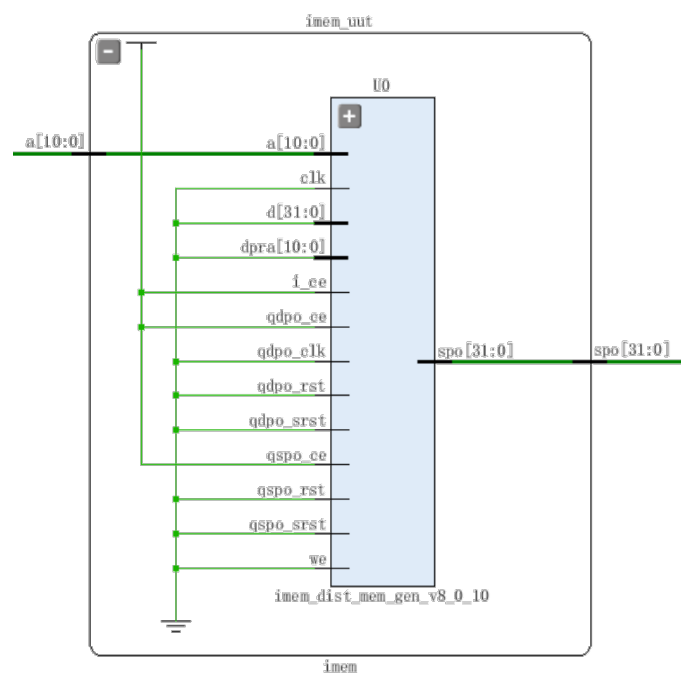
```

② mux 多路选择器模块

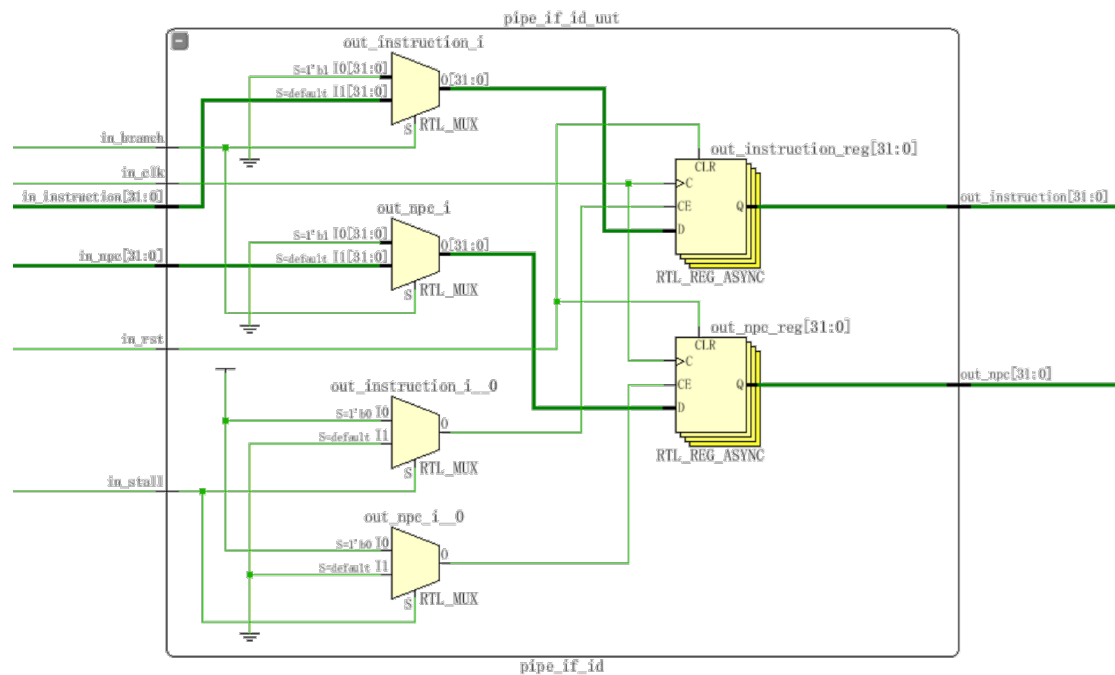


```
// 32 位四选一多路器模块
module mux4_32 (
    input [31:0] d0,    // 输入数据 0
    input [31:0] d1,    // 输入数据 1
    input [31:0] d2,    // 输入数据 2
    input [31:0] d3,    // 输入数据 3
    input [1:0] s,      // 选择信号
    output reg [31:0] y // 输出结果
);
```

③ imem 指令存储器模块（调用 IP 核实现 ROM）



2. IF、ID 段间寄存器段



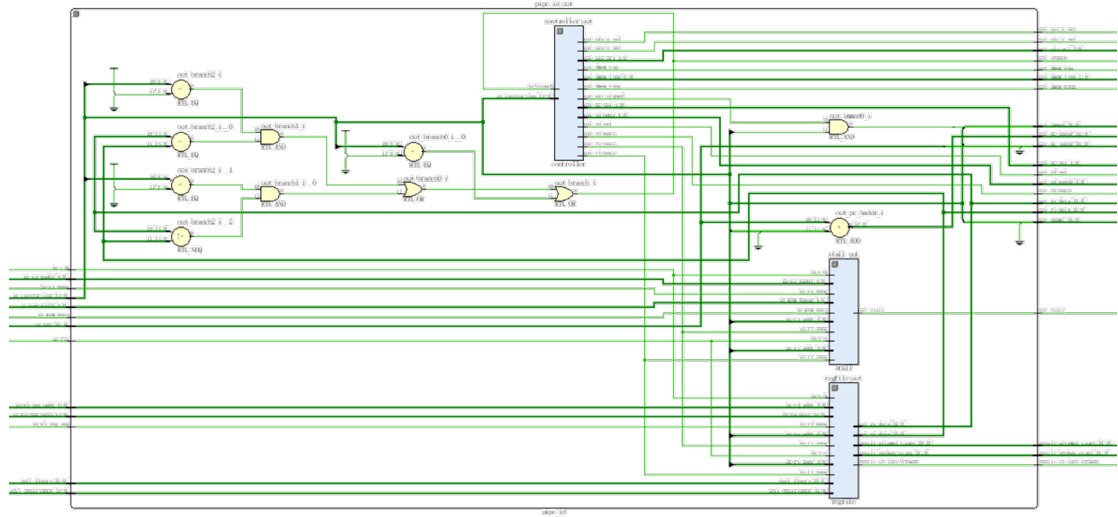
```
// 指令译码流水线寄存器模块
module pipe_if_id(
    input                in_clk,           // 时钟信号
    input                in_rst,           // 复位信号
    input                in_stall,         // 暂停信号
    input                in_branch,        // 分支信号
    input                [31:0] in_npc,    // 下一程序计数器地址
    input                [31:0] in_instruction, // 输入指令
    output reg           [31:0] out_npc,    // 输出下一程序计数器地址
    output reg           [31:0] out_instruction // 输出指令
);
```

该模块是一个流水线寄存器模块，其主要作用是在流水线中存储并传递指令取指阶段的结果。在时钟信号上升沿或者复位信号上升沿触发时，模块进行操作。如果复位信号（`in_rst`）处于高电平状态，表示系统正在进行复位，那么输出寄存器（`out_npc` 和 `out_instruction`）将被置零。

在非暂停状态（`~in_stall`）下，根据分支信号（`in_branch`）的状态来决定是否清零输出寄存器。若分支状态为真，表示处于分支情况，输出寄存器将被清零。否则，将输入的下一程序计数器地址（`in_npc`）和指令（`in_instruction`）分别赋给

输出寄存器（out_npc 和 out_instruction）。在暂停状态下，输出寄存器的值将保持不变。这个模块的设计是为了确保在每个时钟周期内，正确地将译码阶段的输出传递给下一个流水线阶段，以保持流水线的正常运行。

3. IF（指令译码）阶段



```
// 流水线译码模块，负责解析指令和产生控制信号
module pipe_id(
    input                in_clk,                // 时钟信号
    input                in_rst,                // 复位信号

    input  [31:0] in_npc,                      // 当前程序计数器地址
    input  [31:0] in_instruction,              // 输入指令

    input  [4:0] in_ex_waddr,                  // 执行阶段写入地址
    input  [4:0] in_mem_waddr,                // 存储阶段写入地址
    input                in_ex_wena,           // 执行阶段写使能
    input                in_mem_wena,          // 存储阶段写使能

    input  [4:0] in_wb_reg_addr,               // 写回阶段写入寄存器地址
    input                in_wb_reg_ena,        // 写回阶段写入寄存器使能
    input  [31:0] in_wb_reg_data,             // 写回阶段写入寄存器数据

    input  [31:0] init_floors,                 // 初始楼层数
    input  [31:0] init_resistance,            // 初始耐摔值

    output [31:0] out_rs_data,                 // 读取寄存器 rs 数据

```

```

output [31:0] out_rt_data,          // 读取寄存器 rt 数据
output [4:0]  out_rd_waddr,         // 写回阶段写入地址
output       out_rd_sel,           // 写回阶段写入选择信号
output       out_rd_wena,          // 写回阶段写入寄存器使能
output [31:0] out_immed,           // 立即数
output [31:0] out_shamt,           // 移位位数

output       out_dmem_ena,         // 数据存储器读使能
output       out_dmem_wena,        // 数据存储器写使能
output [1:0]  out_dmem_type,       // 数据存储器读写类型

output [31:0] out_pc_baddr,        // 分支时的程序计数器
output [31:0] out_pc_jaddr,        // 跳转时的程序计数器
output [1:0]  out_pc_sel,          // 程序计数器选择信号

output       out_alu_a_sel,         // ALU A 选择信号
output       out_alu_b_sel,         // ALU B 选择信号
output [3:0]  out_alu_sel,          // ALU 操作选择信号

output       out_stall,             // 暂停信号
output       out_branch,           // 分支信号

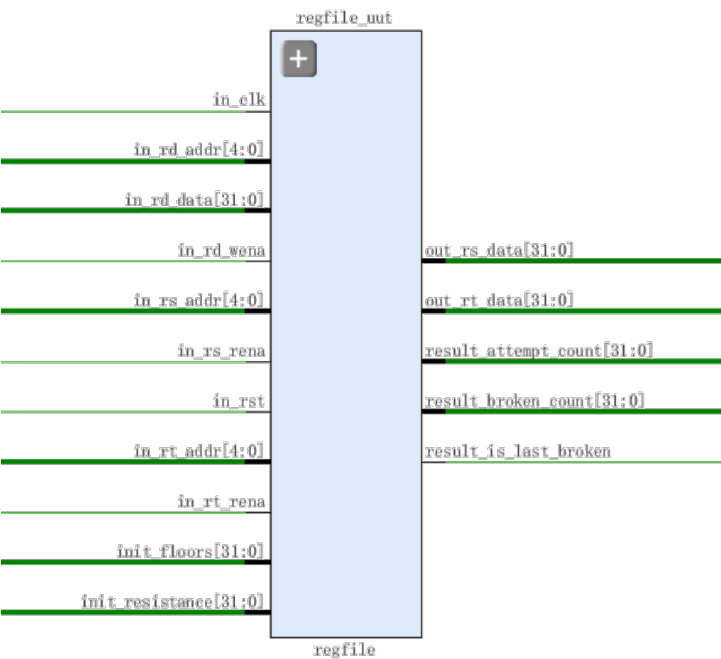
output [31:0] result_attempt_count, // 摔鸡蛋尝试次数
output [31:0] result_broken_count,  // 摔鸡蛋摔破次数
output       result_is_last_broken  // 最后一次是否摔破
);

```

该模块是流水线中的 ID（Instruction Decode）译码模块，主要负责对输入的指令进行解析和控制信号的生成。首先，通过解析指令，提取出关键信息如操作码、功能码以及源、目标寄存器的地址。随后，通过调用寄存器堆模块（regfile）实现对寄存器的读取和写入，获取源寄存器和目标寄存器的数据。通过调用控制器模块（controller），解析当前指令并生成相应的控制信号，如读写使能信号、写入寄存器选择信号等。此外，该模块还生成数据存储器相关的控制信号，包括读写使能信号和读写类型信号。针对 ALU（算术逻辑单元）的操作，通过控制器生成 ALU 的输入选择信号和操作选择信号。为了支持分支和跳转，计算并生成程序计数器的基地址和选择信号。最后，通过调用数据冲突判断模块（stall），判断是否存在数据冲突，从而产生相应的暂停信号。整体而言，该模块在流水线中起到了解析指令、生成控制信号以及处理数据冲突的关键作用，为流水线的下一阶段

段提供了必要的信息和控制。

① regfile 寄存器堆



```
// 寄存器堆模块
module regfile(
    input                in_clk,                // 时钟信号
    input                in_rst,                // 复位信号

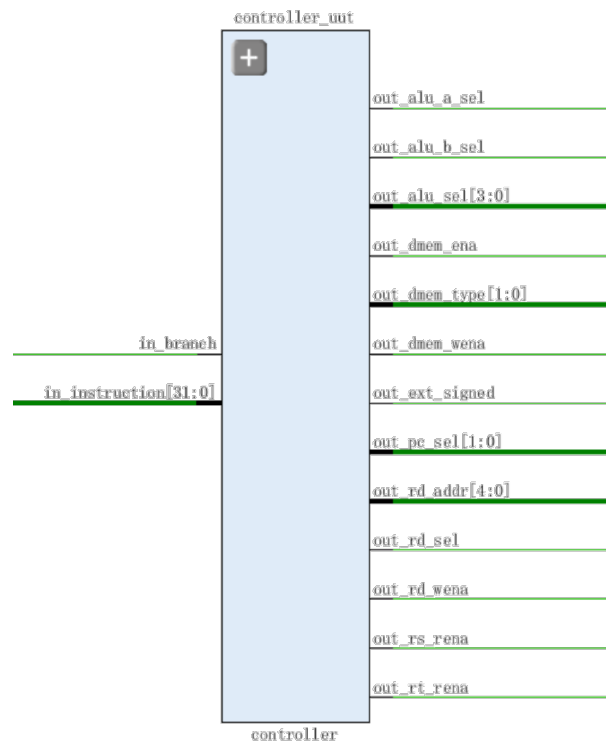
    input                in_rs_rena,            // 源寄存器读使能
    input                in_rt_rena,            // 目标寄存器读使能
    input                in_rd_wena,            // 目标寄存器写使能
    input [4:0]          in_rd_addr,            // 目标寄存器写地址
    input [4:0]          in_rs_addr,            // 源寄存器地址
    input [4:0]          in_rt_addr,            // 目标寄存器地址
    input [31:0]         in_rd_data,            // 目标寄存器写数据

    input [31:0]         init_floors,           // 初始楼层数
    input [31:0]         init_resistance,       // 初始耐摔值

    output reg [31:0]    out_rs_data,           // 源寄存器数据输出
    output reg [31:0]    out_rt_data,           // 目标寄存器数据输出

    output [31:0]        result_attempt_count,  // 摔鸡蛋总次数
    output [31:0]        result_broken_count,   // 摔碎的鸡蛋总数
    output               result_is_last_broken  // 最后是否摔碎
);
```

② controller 控制器



```
// 控制器模块
module controller(
    input          in_branch,           // 分支信号
    input [31:0]   in_instruction,     // 输入指令

    output         out_rs_ena,          // 源寄存器读使能
    output         out_rt_ena,          // 目标寄存器读使能
    output         out_rd_wena,         // 目标寄存器写使能
    output [4:0]   out_rd_addr,         // 目标寄存器写地址
    output         out_rd_sel,          // 目标寄存器写使能

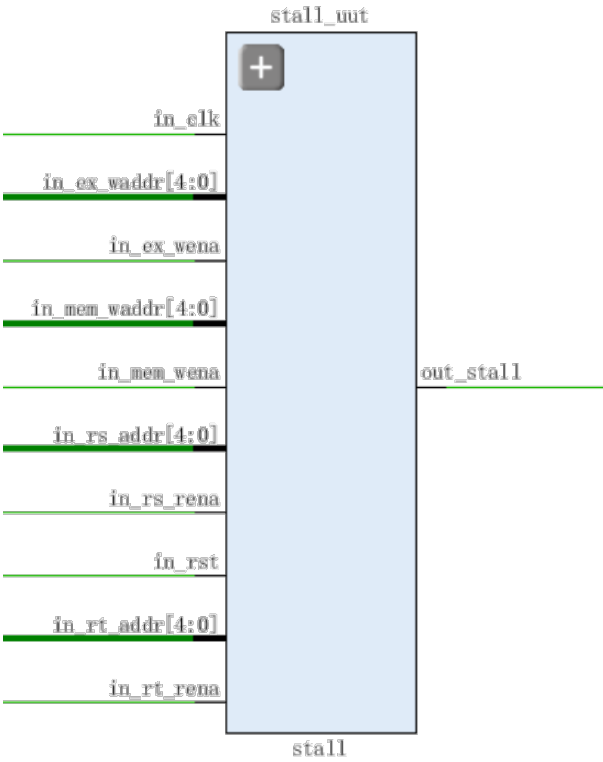
    output         out_dmem_ena,        // 数据存储器读使能
    output         out_dmem_wena,       // 数据存储器写使能
    output [1:0]   out_dmem_type,       // 数据存储器写类型

    output         out_ext_signed,      // 符号扩展信号
    output         out_alu_a_sel,       // ALU A 选择信号
    output         out_alu_b_sel,       // ALU B 选择信号
    output [3:0]   out_alu_sel,         // ALU 控制信号
    output [1:0]   out_pc_sel           // PC 选择信号
);
```

该模块是流水线中的控制器模块，主要功能是解析输入的指令并产生对应的控制信号，以驱动流水线的各个阶段的操作。通过检测输入指令的操作码和功能码，该模块确定指令的类型并生成相应的控制信号。控制信号包括源寄存器读使能、目标寄存器读使能、目标寄存器写使能、目标寄存器写地址、目标寄存器写使能、数据存储器读使能、数据存储器写使能、数据存储器写类型、符号扩展信号、ALU A/B 选择信号、ALU 控制信号、PC 选择信号等。

该模块通过逻辑比较和组合逻辑的方式判断当前指令的类型，并生成相应的控制信号，这些信号将用于控制其他模块的行为，协调流水线的运行。

③ **stall 暂停信号模块**



```
// 流水线中的暂停模块，用于检测并处理数据冲突，实现对应的暂停机制。
module stall(
    input          in_clk,          // 时钟信号
    input          in_rst,          // 复位信号

    input  [4:0]   in_rs_addr,      // 源寄存器地址
    input  [4:0]   in_rt_addr,      // 目标寄存器地址
    input          in_rs_rena,      // 源寄存器读使能
    input          in_rt_rena,      // 目标寄存器读使能

```

```

input          in_ex_wena,      // EX 阶段写使能
input          in_mem_wena,     // MEM 阶段写使能
input  [4:0]    in_ex_waddr,    // EX 阶段写地址
input  [4:0]    in_mem_waddr,   // MEM 阶段写地址

output reg      out_stall       // 输出的暂停信号
);

```

stall 模块的机制是为了解决流水线中可能发生的数据冲突导致的暂停情况。在每个时钟周期的下降沿，模块会检查当前是否需要暂停流水线。

在正常运行状态下，模块会检查数据冲突情况。如果存在 ex 阶段与 id 阶段之间的读写冲突，即 ex 阶段需要写入的目标寄存器与 id 阶段的源寄存器之一相同，模块会将 stall_ltime 设置为 1，表示暂停流水线两个周期。另一方面，如果存在 ex 阶段与 mem 阶段之间的读写冲突，即 ex 阶段需要写入的目标寄存器与 mem 阶段的目标寄存器相同，模块会将 stall_ltime 设置为 0（配合 out_stall=1），表示暂停流水线一个周期。在暂停期间，计数器 stall_ltime 会递减，直至减至零。此时，流水线可以继续正常运行。整个机制通过控制 out_stall 信号，实现了在发生数据冲突时暂停流水线，确保数据的正确传递和处理。

4. ID、EX 段间寄存段



// 指令译码到执行流水线寄存器模块

```
module pipe_id_ex(
    input          in_clk,          // 时钟信号
    input          in_rst,          // 复位信号

    input          in_dmem_ena,      // 数据存储器使能信号
    input          in_dmem_wena,     // 数据存储器写使能信号
    input [1:0]    in_dmem_type,     // 数据存储器访问类型

    input [31:0]   in_rs_data,       // 源寄存器数据
    input [31:0]   in_rt_data,       // 目标寄存器数据
    input [4:0]    in_rd_waddr,      // 写目标寄存器地址
    input          in_rd_sel,        // 写目标寄存器选择信号
    input          in_rd_wena,       // 写目标寄存器使能信号

    input [31:0]   in_immed,         // 立即数
    input [31:0]   in_shamt,         // 移位操作位数

    input          in_alu_a_sel,     // ALU 操作数 A 选择信号
    input          in_alu_b_sel,     // ALU 操作数 B 选择信号
    input [3:0]    in_alu_sel,       // ALU 操作选择信号

    input          in_stall,         // 流水线是否暂停信号

    output reg     out_dmem_ena,     // 输出数据存储器使能信号
    output reg     out_dmem_wena,    // 输出数据存储器写使能信号
    output reg [1:0] out_dmem_type,  // 输出数据存储器访问类型

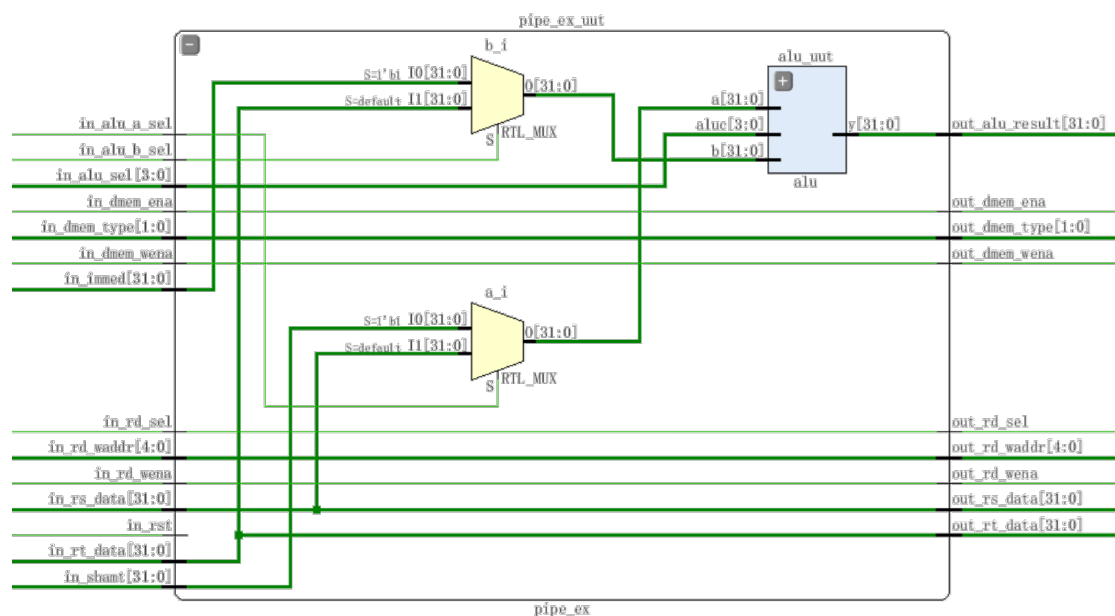
    output reg [31:0] out_rs_data,   // 输出源寄存器数据
    output reg [31:0] out_rt_data,   // 输出目标寄存器数据
    output reg [4:0]  out_rd_waddr,  // 输出写目标寄存器地址
    output reg        out_rd_sel,    // 输出写目标寄存器选择信号
    output reg        out_rd_wena,   // 输出写目标寄存器使能信号

    output reg [31:0] out_immed,     // 输出立即数
    output reg [31:0] out_shamt,     // 输出移位操作位数

    output reg      out_alu_a_sel,   // 输出 ALU 操作数 A 选择信号
    output reg      out_alu_b_sel,   // 输出 ALU 操作数 B 选择信号
    output reg [3:0] out_alu_sel     // 输出 ALU 操作选择信号
);
```

该模块是流水线的 ID（译码）到 EX（执行）阶段的寄存器模块。在时钟上升沿或复位信号上升沿时，根据输入的控制信号和数据，更新流水线的输出信号，为下一个阶段的执行提供必要的输入。模块中包含了对输入信号的判断和处理逻辑，以及根据输入更新输出的逻辑。具体功能包括控制数据存储器的使能信号、写使能信号和访问类型的输出；输出源寄存器数据、目标寄存器数据、写目标寄存器地址、写目标寄存器选择信号和写目标寄存器使能信号；输出立即数和移位操作位数；以及控制 ALU 操作数 A 和 B 选择的信号，以及 ALU 操作选择的信号。此外，模块还考虑了流水线是否处于暂停状态，如果是，则清空输出信号，保持流水线的同步和正确性。该模块能够将控制信号和数据在流水线中传递，为指令的执行提供必要的数据和控制。

5. EX（执行）阶段



```
// 流水线执行阶段模块
module pipe_ex(
    input                in_rst,

    input                in_dmem_ena,    // 数据存储器使能信号
    input                in_dmem_wena,    // 数据存储器写使能信号
    input [1:0]          in_dmem_type,    // 数据存储器访问类型

    input [31:0]         in_rs_data,      // 输入源寄存器数据
    input [31:0]         in_rt_data,      // 输入目标寄存器数据
```



```

input    [4:0]    in_rd_waddr,      // 输入写目标寄存器地址
input    in_rd_sel,      // 输入写目标寄存器选择信号
input    in_rd_wena,      // 输入写目标寄存器使能信号

input    [31:0]   in_immed,      // 输入立即数
input    [31:0]   in_shamt,      // 输入移位操作位数

input    in_alu_a_sel,      // 输入 ALU 操作数 A 选择信号
input    in_alu_b_sel,      // 输入 ALU 操作数 B 选择信号
input    [3:0]    in_alu_sel,      // 输入 ALU 操作选择信号

output    out_dmem_ena,      // 输出数据存储器使能信号
output    out_dmem_wena,      // 输出数据存储器写使能信号
output    [1:0]   out_dmem_type,  // 输出数据存储器访问类型

output    [31:0]   out_rs_data,    // 输出源寄存器数据
output    [31:0]   out_rt_data,    // 输出目标寄存器数据
output    [4:0]    out_rd_waddr,    // 输出写目标寄存器地址
output    out_rd_sel,      // 输出写目标寄存器选择信号
output    out_rd_wena,      // 输出写目标寄存器使能信号

output    [31:0]   out_alu_result  // 输出 ALU 运算结果
);

```

该模块是流水线执行阶段（pipe_ex）的实现，主要负责控制 ALU（算术逻辑单元）的运算。在 ALU 运算控制方面，模块接收 ALU 的两个操作数（in_alu_a_sel、in_alu_b_sel、in_alu_sel）进行运算，并输出结果（out_alu_result）。ALU 运算数选择方面，输入的 ALU 运算数根据选择信号（in_alu_a_sel、in_alu_b_sel）进行选择，可以是寄存器数据或者立即数（immediate 或 shamt）。综上所述，模块实现了流水线执行阶段对指令的计算和对数据存储器的操作，为流水线的正常运行提供了必要的数据和控制信号。

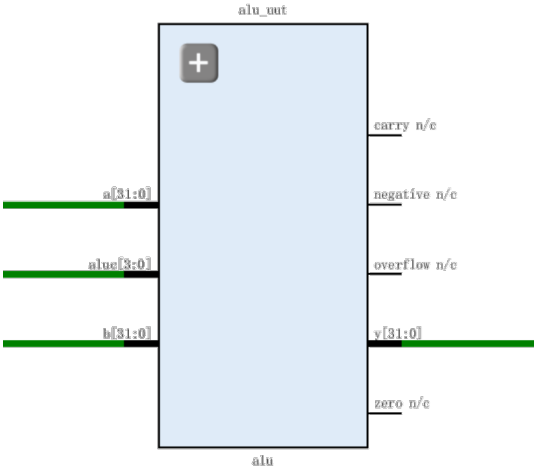
① ALU 模块

```

// 算术逻辑单元模块
module alu(
    input    [31:0]   a,          // 输入操作数 a
    input    [31:0]   b,          // 输入操作数 b
    output    [31:0]   y,          // 输出结果

```

```
input  [3:0]  aluc,      // ALU 控制信号
output          zero,    // 零标志位
output          carry,    // 进位标志位
output          negative, // 负数标志位
output          overflow  // 溢出标志位
);
```



ALU 对应的操作信号如下所示：

	ALU_SEL[3]	ALU_SEL[2]	ALU_SEL[1]	ALU_SEL[0]
ADD	0	0	0	0
ADDU	0	0	0	1
SUB	0	0	1	0
SUBU	0	0	1	1
AND	0	1	0	0
OR	0	1	0	1
XOR	0	1	1	0
NOR	0	1	1	1
SLT	1	0	0	0
SLTU	1	0	0	1
SLL	1	0	1	0
SRL	1	0	1	1

SRA	1	1	0	0
LUI	1	1	0	1

6. EX、MEM 段间寄存段



```
// 执行、访存模块间寄存器模块
module pipe_ex_mem(
    input                in_clk,
    input                in_rst,

    input                in_dmem_ena,    // 数据存储器使能信号
    input                in_dmem_wena,   // 数据存储器写使能信号
    input [1:0]          in_dmem_type,   // 数据存储器类型

    input [31:0]          in_rs_data,    // 寄存器数据
    input [31:0]          in_rt_data,    // 寄存器数据
    input [4:0]           in_rd_waddr,    // 目标寄存器地址
    input                in_rd_sel,      // 目标寄存器选择信号
    input                in_rd_wena,     // 目标寄存器写使能信号

    input [31:0]          in_alu_result, // ALU 运算结果

    output reg           out_dmem_ena,   // 输出数据存储器使能信号

```

```

output reg          out_dmem_wena,    // 输出数据存储器写使能信号
output reg [1:0]    out_dmem_type,    // 输出数据存储器类型

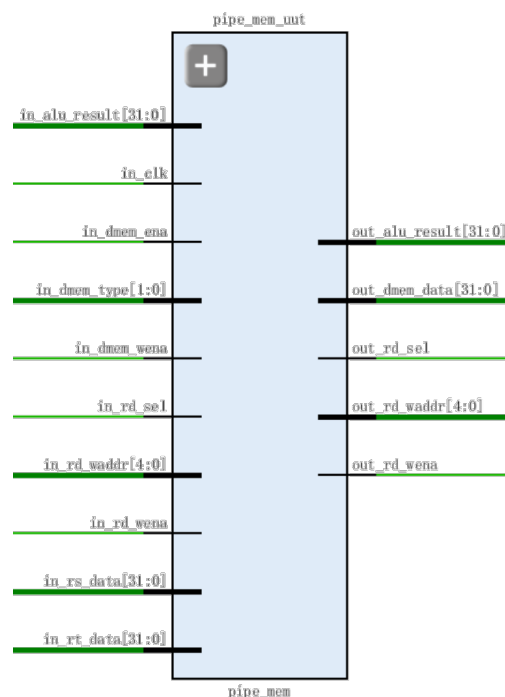
output reg [31:0]   out_rs_data,      // 输出寄存器数据
output reg [31:0]   out_rt_data,      // 输出寄存器数据
output reg [4:0]    out_rd_waddr,     // 输出目标寄存器地址
output reg          out_rd_sel,       // 输出目标寄存器选择信号
output reg          out_rd_wena,      // 输出目标寄存器写使能信号

output reg [31:0]   out_alu_result    // 输出 ALU 运算结果
);

```

上述模块充当了流水线中执行阶段（EX）和访存阶段（MEM）之间的寄存器，负责传递控制信号和计算结果。其主要功能是在时钟上升沿或复位信号触发时，根据输入信号的状态，将数据存储器（Data Memory）的使能信号、写使能信号和类型信号传递到下一阶段，同时传递寄存器相关的数据、写地址、写选择信号和写使能信号，以及 ALU 运算的结果。在复位状态下，所有输出信号被置零；在正常运行状态下，输入信号直接传递给对应的输出信号，即完成寄存一个周期的功能。这有助于确保流水线的正确运行，保持数据的传递一致性，同时提供对数据存储器和寄存器的控制和写入。

7. MEM（访存）阶段



```

// 访存阶段模块
module pipe_mem(
    input          in_clk,          // 时钟信号

    input          in_dmem_ena,      // 数据存储器使能信号
    input          in_dmem_wena,     // 数据存储器写使能信号
    input  [1:0]    in_dmem_type,    // 数据存储器类型信号

    input  [31:0]   in_rs_data,      // 执行阶段传递的寄存器数据
    input  [31:0]   in_rt_data,      // 执行阶段传递的寄存器数据
    input  [4:0]    in_rd_waddr,     // 执行阶段传递的写地址
    input          in_rd_sel,        // 执行阶段传递的写选择信号
    input          in_rd_wena,       // 执行阶段传递的写使能信号

    input  [31:0]   in_alu_result,   // 执行阶段传递的 ALU 计算结果

    output [4:0]    out_rd_waddr,     // 输出到写回阶段的写地址
    output          out_rd_sel,       // 输出到写回阶段的写选择信号
    output          out_rd_wena,      // 输出到写回阶段的写使能信号

    output [31:0]   out_alu_result,   // 输出到写回阶段的 ALU 计算结果
    output [31:0]   out_dmem_data    // 输出到数据存储器的数据
);

```

该模块是流水线中的访存阶段（MEM）模块。其功能主要包括：接收来自执行阶段的输入信号，包括寄存器数据、ALU 计算结果等；控制数据存储器的读写操作，通过传递给数据存储器的输入信号实现；将访存阶段计算的结果传递给写回阶段，包括 ALU 计算结果和数据存储器的读取数据；将写回阶段的写使能信号、写选择信号和写地址传递给下一阶段。总体而言，该模块负责执行访存阶段的操作，包括数据存储器的读写和与写回阶段的数据传递。

① dmem 模块

```

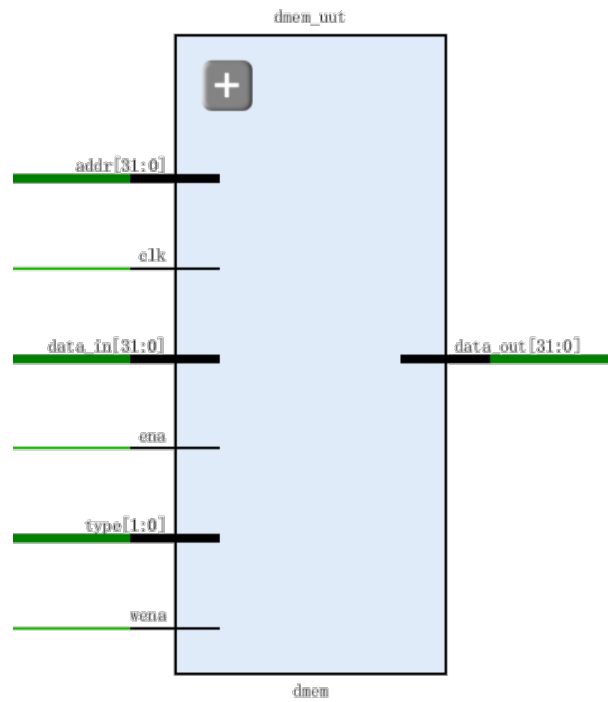
// 数据存储器模块
module dmem(
    input          clk,             // 时钟输入
    input          ena,             // 使能信号
    input          wena,            // 写使能信号
    input  [31:0]   addr,           // 地址信号

```

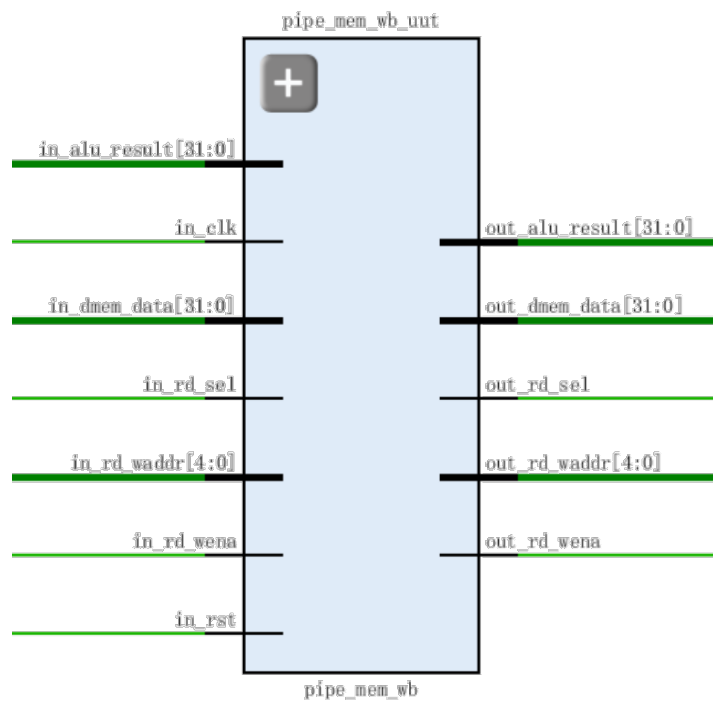
```

input  [1:0]  dmem_type,    // 数据存储类型
input  [31:0] data_in,      // 写入数据
output [31:0] data_out      // 读取数据输出
);

```



8. MEM、WB 段间寄存段



```

// 内存访问和写回间的寄存模块
module pipe_mem_wb(
    input          in_clk,          // 时钟信号
    input          in_rst,          // 复位信号

    input          [4:0] in_rd_waddr, // 读取寄存器地址
    input          in_rd_sel,        // 读取寄存器选择信号
    input          in_rd_wena,       // 读取寄存器写使能信号

    input          [31:0] in_alu_result, // ALU 计算结果
    input          [31:0] in_dmem_data,  // 数据存储器读取数据

    output reg     [4:0] out_rd_waddr, // 读取寄存器地址输出
    output reg     out_rd_wena,        // 读取寄存器写使能输出
    output reg     out_rd_sel,         // 读取寄存器选择输出

    output reg     [31:0] out_alu_result, // ALU 计算结果输出
    output reg     [31:0] out_dmem_data  // 数据存储器读取数据输出
);

```

该模块是内存访问（MEM）和写回（WB）两个阶段之间的寄存器模块。其主要功能是在时钟上升沿或复位信号触发时，根据输入信号的状态更新输出信号。模块内包含了读取寄存器地址、读取寄存器选择、读取寄存器写使能、ALU 计算结果和数据存储器读取数据等输入信号。在每个时钟周期内，根据复位信号的状态，模块将输入信号的值传递给相应的输出信号，以保持流水线的数据传输。这有助于协调 MEM 和 WB 两个阶段的数据传输，确保数据正确流经流水线的各个阶段，完成指令的执行。

9. WB（写回）阶段

```

// 写回阶段模块
module pipe_wb(
    input          [4:0] in_rd_waddr, // 写回寄存器地址
    input          in_rd_wena,        // 写回寄存器写使能
    input          in_rd_sel,         // 写回寄存器选择

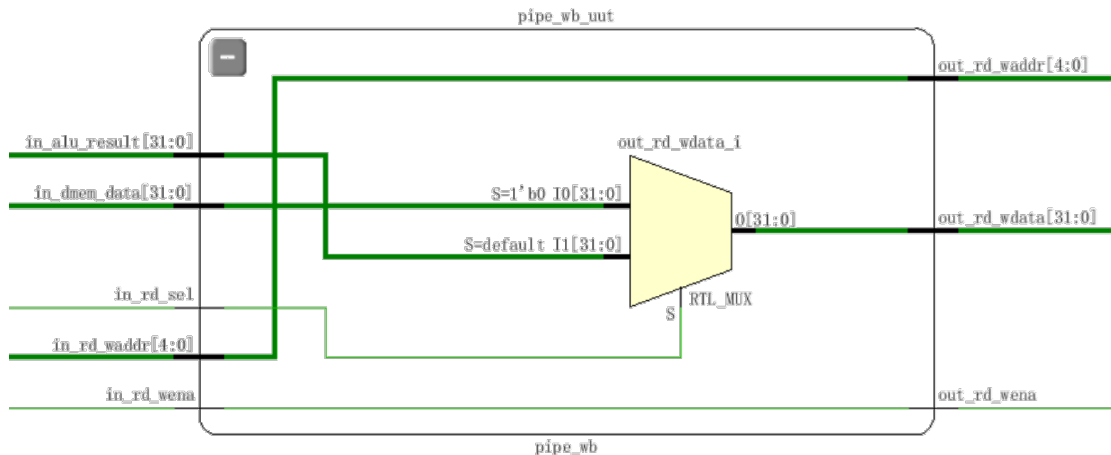
    input          [31:0] in_alu_result, // ALU 计算结果
    input          [31:0] in_dmem_data,  // 数据存储器读取数据

```

```

output [4:0] out_rd_waddr, // 写回寄存器地址
output      out_rd_wena,  // 写回寄存器写使能
output [31:0] out_rd_wdata // 写回寄存器写入数据
);

```



该模块是数据通路中 Write Back（WB）阶段的寄存器模块。该模块接收来自 MEM 阶段传递来的 ALU 计算结果或内存数据，根据控制信号选择将哪一个数据写入寄存器文件。具体而言，当写回使能信号（in_rd_wena）为高时，根据写回选择信号（in_rd_sel）的不同，选择写入 ALU 计算结果或数据存储器读取的数据。同时，将写入的寄存器地址（in_rd_waddr）输出到 regfile 写入。

四、实验仿真过程

1. 构建仿真测试指令

各条指令的仿真测试随 CPU 模块的搭建共同进行，较为繁琐，因此此处不进行赘述。最终进行仿真时，选用了第六部分中所阐述的比萨塔摔鸡蛋程序，转为 coe 文件后初始化 imem 的 IP 核。

2. 撰写 testbench 程序

为了在一次下板过程中进行多组数据的调试，初始化的层高与耐摔值从外层模块传入。用于 reset 时直接初始化对应的 regfile 寄存器。testbench 程序如下，可以将每个 clk 时的 cpu 值输出到 log 文件中。


```

module testbench(

);

reg        clk;
reg        rst;

reg [15:0] init_data;
reg        is_init_floors;
reg        is_init_resistance;
wire       last_broken;

integer     regfile_output;

initial
begin
    regfile_output = $fopen("regfile_output.txt");
    clk = 0;
    rst = 1;
    #20 init_data = 16'd0030;    // 初始化楼高
    #20 is_init_floors = 1'b1;
    #20 is_init_floors = 1'b0;
    #20 init_data = 16'd0019;    // 初始化耐摔值
    #20 is_init_resistance = 1'b1;
    #20 is_init_resistance = 1'b0;
    #20 rst = 0;
end

always
begin
    #5 clk = ~clk;
end

// .....省略 log 信息的输出.....

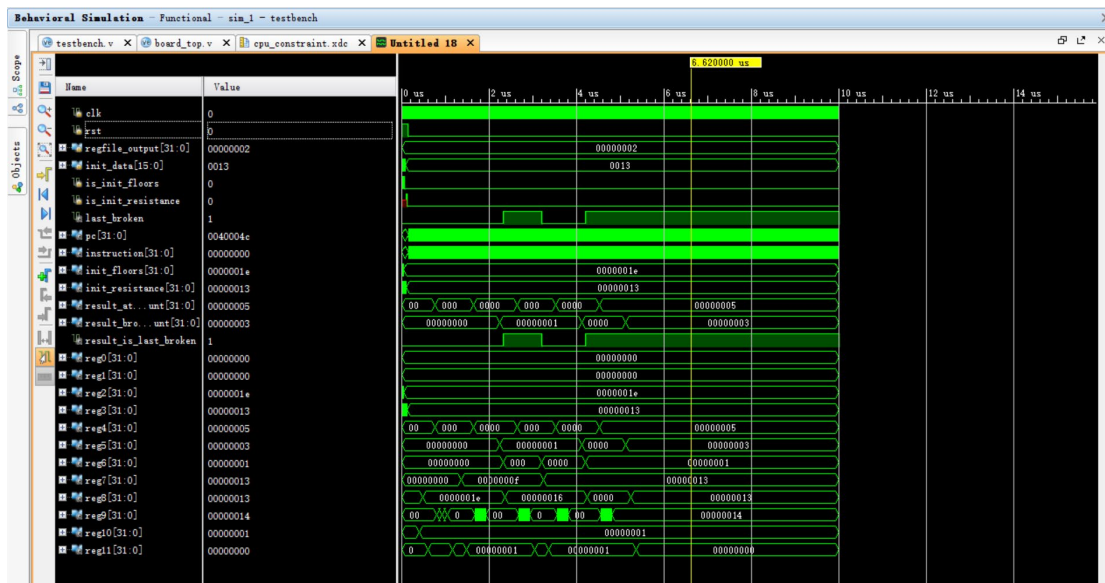
board_top board_top_uut(
    .in_clk(clk),
    .in_rst(rst),
    .in_data(init_data),
    .is_init_floors(is_init_floors),
    .is_init_resistance(is_init_resistance),
    .result_is_last_broken(last_broken)
);

endmodule

```

3. 运行仿真

如上述 testbench 文件所述，测试初始化楼层高度为 30，鸡蛋耐摔值为 19 时的情况。在 Vivado 中进行仿真，可以观察到寄存器当中的值与波形图。与 MARS 运行的 asm 程序比较，可见寄存器中值均相同；与 C 语言的运行结果也一致。



Registers	Coproc 1	Coproc 0
Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x00000000
\$v0	2	0x0000001e
\$v1	3	0x00000013
\$a0	4	0x00000005
\$a1	5	0x00000003
\$a2	6	0x00000001
\$a3	7	0x00000013
\$t0	8	0x00000013
\$t1	9	0x00000014
\$t2	10	0x00000001

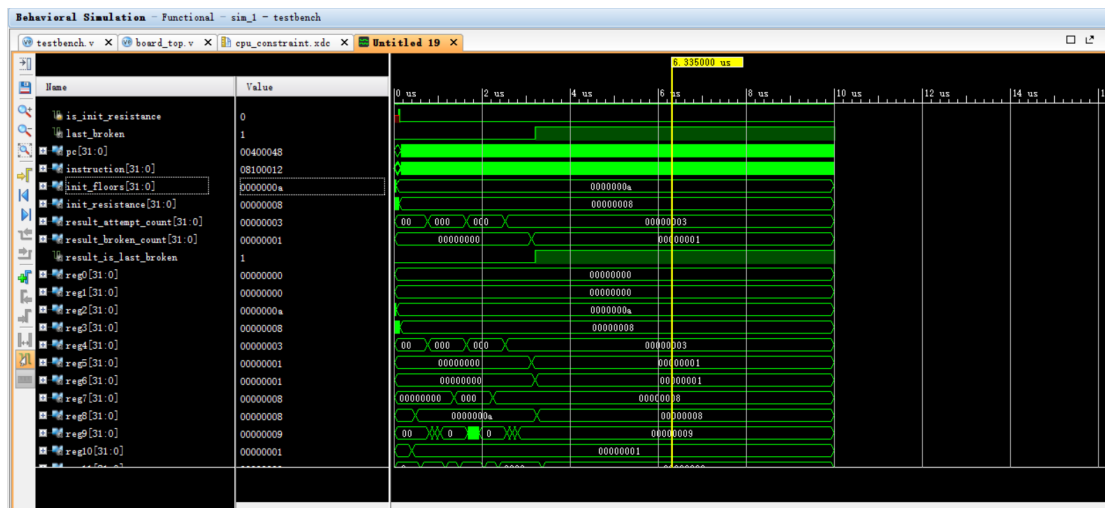
```

Microsoft Visual Studio 调试
请输入楼层数：30
请输入耐摔值：19
进行第1次实验，在15层扔鸡蛋，本次鸡蛋没碎
进行第2次实验，在23层扔鸡蛋，本次鸡蛋摔碎
进行第3次实验，在19层扔鸡蛋，本次鸡蛋没碎
进行第4次实验，在21层扔鸡蛋，本次鸡蛋摔碎
进行第5次实验，在20层扔鸡蛋，本次鸡蛋摔碎
共尝试5次，摔碎鸡蛋3个，最后一次鸡蛋摔碎
  
```

五、实验仿真的波形图及某时刻寄存器值的物理意义

1. 流水线波形图

测试初始化楼层高度为 10，鸡蛋耐摔值为 8 时的情况，得到波形图如下：



2. 某时刻寄存器值的物理意义

选择 $t=6335000\mu\text{s}$ 时刻，分析各个寄存器的值的含义。此时，PC 寄存器的值为 $0x400048$ ，对应 END 标签。程序已经进入 END 处的死循环，正确的结果已经被保存至响应的寄存器。

\$2 号寄存器的值为 10，\$3 号寄存器的值为 8，对应初始化输入时的楼层总高度核鸡蛋耐摔值。\$10 号寄存器通过第一条指令被初始化为常数 1，用于之后的比较。\$7 号与\$8 号寄存器分别存放二分算法的左边界 l 和右边界 r ，此时相等均为 8。\$4 号寄存器的值为 3，代表需要进行 3 次实验；\$5 号寄存器的值为 1，代表摔碎了 1 个鸡蛋；\$6 号寄存器的值为 0，代表最后一次鸡蛋没有摔碎。上述通过波形图得到的寄存器值与 MARS 中汇编程序运行结果一致：

Registers	Coproc 1	Coproc 0	
Name	Number	Value	
\$zero	0	0x00000000	
\$at	1	0x00000000	
\$v0	2	0x0000000a	
\$v1	3	0x00000008	
\$a0	4	0x00000003	
\$a1	5	0x00000001	
\$a2	6	0x00000001	
\$a3	7	0x00000008	
\$t0	8	0x00000008	
\$t1	9	0x00000009	
\$t2	10	0x00000001	

六、流水线 CPU 实验性能验证模型

1. 实验性能验证模型

比萨塔摔鸡蛋游戏。两个同学在可变换层数的比萨塔上摔鸡蛋，一个同学秘密设定同一批鸡蛋耐摔值；另一个同学在指定层高的比萨塔拿着鸡蛋往下摔，用最少的摔次数和摔破的鸡蛋数求出鸡蛋的耐摔值。假定在耐摔值的楼层及其下面楼层，鸡蛋摔不破，可以重复使用，否则鸡蛋摔破。要求模型的算法输出包括：摔的总次数、摔的总鸡蛋数、最后摔的鸡蛋是否摔破。请使用 C 语言设计该验证模型的算法，并把 C 语言汇编为 MIPS 指令汇编程序，同时利用编译器生成 MIPS 指令集可执行目标程序。

2. C 语言验证程序

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

int main()
{
    int init_floor;
    int init_resistance;

    printf("请输入楼层数: ");
    scanf("%d", &init_floor);

    printf("请输入耐摔值: ");
    scanf("%d", &init_resistance);

    int result_attempt_cnt = 0;
    int result_broken_cnt = 0;
    bool result_is_last_broken = 0;

    int l = 0, r = init_floor;

    while (l < r) {

        result_attempt_cnt++;

        int mid = (l + r + 1) / 2;

        if (mid > init_resistance) {
```

```

        result_is_last_broken = 1;
        r = mid - 1;
        result_broken_cnt ++;
    }
    else{
        result_is_last_broken = 0;
        l = mid;
    }
    printf("进行第%d次实验，在%d层扔鸡蛋，本次鸡蛋", result_attempt_cnt, mid);
    printf(result_is_last_broken ? "摔碎\n" : "没碎\n");
}

    printf("共尝试%d次，摔碎鸡蛋%d个，最后一次鸡蛋", result_attempt_cnt, result_broken_cnt );
    printf((result_is_last_broken ? "摔碎\n" : "没碎\n"));

    return 0;
}

```

3. MIPS 指令汇编验证程序

```

# 比萨斜塔摔鸡蛋ASM汇编程序

# 寄存器用途说明：
# $2 (楼层上限)：建筑物的最大楼层数
# $3 (鸡蛋耐摔值)：鸡蛋的最大可忍受摔落次数
# $4 (总摔鸡蛋次数)：记录总共的摔鸡蛋次数
# $5 (碎裂的鸡蛋数)：记录摔碎的鸡蛋数
# $6 (最后一个鸡蛋是否碎裂)：标志位，1 表示鸡蛋碎裂，0 表示未碎裂
# $7 (左边搜索边界)：二分查找时的左边搜索边界
# $8 (右边搜索边界)：二分查找时的右边搜索边界
# $9 (中间值)：二分查找时的中间值
# $10：常数 1

.text
addi $2, $0, 9      # 设置楼层上限 n
addi $3, $0, 3      # 设置鸡蛋耐摔值 m
addi $10, $0, 1     # 常数 1 ($10=1)
add $7, $0, $0      # 左边界 l=0
add $8, $0, $2      # 右边界 r=n

```

```

LOOP:
slt $11, $7, $8      # 检查左边界是否大于等于右边界 ($11=0)
beq $11, $0, END     # 如果左边界大于等于右边界, 跳转到 END ($11=0)
addi $4, $4, 1       # 增加总摔鸡蛋次数
add $9, $7, $8
addi $9, $9, 1

sra $9, $9, 1        # 计算中间值 mid = (l+r+1)/2
slt $11, $3, $9      # 检查鸡蛋耐摔值是否大于等于中间值 ($11=0)
beq $11, $10, BROKE  # 如果鸡蛋耐摔值大于等于中间值, 跳转到 BROKE ($11=
=1)
RESISIT:
add $6, $0, $0       # 设置最后一个鸡蛋是否碎裂为 0
add $7, $0, $9       # 更新左边界为中间值
j LOOP

BROKE:
addi $5, $5, 1       # 增加碎裂的鸡蛋数
add $6, $0, $10      # 设置最后一个鸡蛋是否碎裂为 1
sub $8, $9, $10      # 更新右边界为中间值减一
j LOOP

END:
j END                # 无限循环

.text ends

```

七、实验验算程序下板测试过程与实现

1. 管脚约束文件

```

set_property PACKAGE_PIN E3 [get_ports in_clk]
set_property PACKAGE_PIN N17 [get_ports in_rst]
set_property PACKAGE_PIN P17 [get_ports is_init_floors]
set_property PACKAGE_PIN M17 [get_ports is_init_resistance]
set_property PACKAGE_PIN H17 [get_ports result_is_last_broken]

set_property PACKAGE_PIN T10 [get_ports {o_seg[0]}]
set_property PACKAGE_PIN R10 [get_ports {o_seg[1]}]
set_property PACKAGE_PIN K16 [get_ports {o_seg[2]}]
set_property PACKAGE_PIN K13 [get_ports {o_seg[3]}]
set_property PACKAGE_PIN P15 [get_ports {o_seg[4]}]
set_property PACKAGE_PIN T11 [get_ports {o_seg[5]}]
set_property PACKAGE_PIN L18 [get_ports {o_seg[6]}]
set_property PACKAGE_PIN H15 [get_ports {o_seg[7]}]

```

```

set_property PACKAGE_PIN J17 [get_ports {o_sel[0]}]
set_property PACKAGE_PIN J18 [get_ports {o_sel[1]}]
set_property PACKAGE_PIN T9 [get_ports {o_sel[2]}]
set_property PACKAGE_PIN J14 [get_ports {o_sel[3]}]
set_property PACKAGE_PIN P14 [get_ports {o_sel[4]}]
set_property PACKAGE_PIN T14 [get_ports {o_sel[5]}]
set_property PACKAGE_PIN K2 [get_ports {o_sel[6]}]
set_property PACKAGE_PIN U13 [get_ports {o_sel[7]}]

set_property PACKAGE_PIN J15 [get_ports {in_data[0]}]
set_property PACKAGE_PIN L16 [get_ports {in_data[1]}]
set_property PACKAGE_PIN M13 [get_ports {in_data[2]}]
set_property PACKAGE_PIN R15 [get_ports {in_data[3]}]
set_property PACKAGE_PIN R17 [get_ports {in_data[4]}]
set_property PACKAGE_PIN T18 [get_ports {in_data[5]}]
set_property PACKAGE_PIN U18 [get_ports {in_data[6]}]
set_property PACKAGE_PIN R13 [get_ports {in_data[7]}]
set_property PACKAGE_PIN T8 [get_ports {in_data[8]}]
set_property PACKAGE_PIN U8 [get_ports {in_data[9]}]
set_property PACKAGE_PIN R16 [get_ports {in_data[10]}]
set_property PACKAGE_PIN T13 [get_ports {in_data[11]}]
set_property PACKAGE_PIN H6 [get_ports {in_data[12]}]
set_property PACKAGE_PIN U12 [get_ports {in_data[13]}]
set_property PACKAGE_PIN U11 [get_ports {in_data[14]}]
set_property PACKAGE_PIN V10 [get_ports {in_data[15]}]

set_property IOSTANDARD LVCMOS33 [get_ports in_clk]
set_property IOSTANDARD LVCMOS33 [get_ports in_rst]
set_property IOSTANDARD LVCMOS33 [get_ports is_init_floors]
set_property IOSTANDARD LVCMOS33 [get_ports is_init_resistance]
set_property IOSTANDARD LVCMOS33 [get_ports
result_is_last_broken]

set_property IOSTANDARD LVCMOS33 [get_ports {o_sel[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {o_sel[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {o_sel[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {o_sel[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {o_sel[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {o_sel[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {o_sel[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {o_sel[0]}]

set_property IOSTANDARD LVCMOS33 [get_ports {o_seg[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {o_seg[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {o_seg[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {o_seg[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {o_seg[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {o_seg[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {o_seg[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {o_seg[0]}]

set_property IOSTANDARD LVCMOS33 [get_ports {in_data[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {in_data[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {in_data[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {in_data[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {in_data[4]}]

```



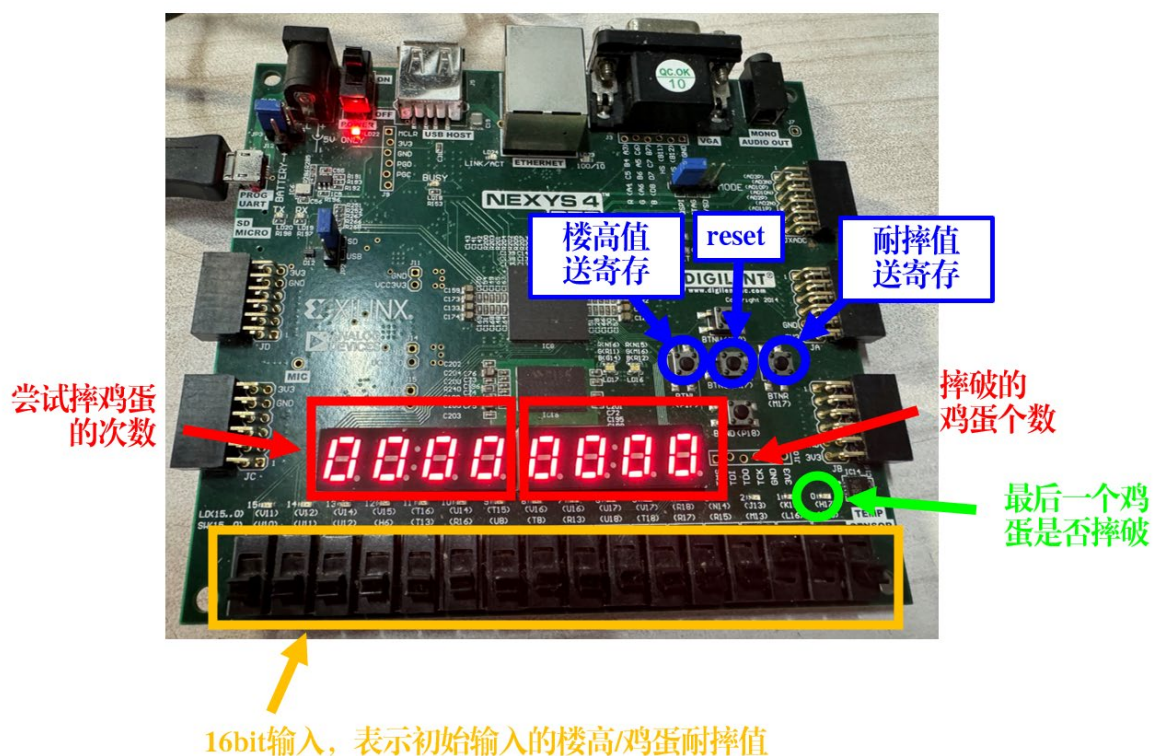
```

set_property IOSTANDARD LVCMOS33 [get_ports {in_data[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {in_data[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {in_data[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {in_data[8]}]
set_property IOSTANDARD LVCMOS33 [get_ports {in_data[9]}]
set_property IOSTANDARD LVCMOS33 [get_ports {in_data[10]}]
set_property IOSTANDARD LVCMOS33 [get_ports {in_data[11]}]
set_property IOSTANDARD LVCMOS33 [get_ports {in_data[12]}]
set_property IOSTANDARD LVCMOS33 [get_ports {in_data[13]}]
set_property IOSTANDARD LVCMOS33 [get_ports {in_data[14]}]
set_property IOSTANDARD LVCMOS33 [get_ports {in_data[15]}]

create_clock -period 100.000 -name clk_pin -waveform {0.000
50.000} [get_ports in_clk]
set_input_delay -clock [get_clocks *] 1.000 [get_ports reset]
set_output_delay -clock [get_clocks *] 0.000 [get_ports -filter
{ NAME =~ "*" && DIRECTION == "OUT" }]

```

2. 下板测试过程

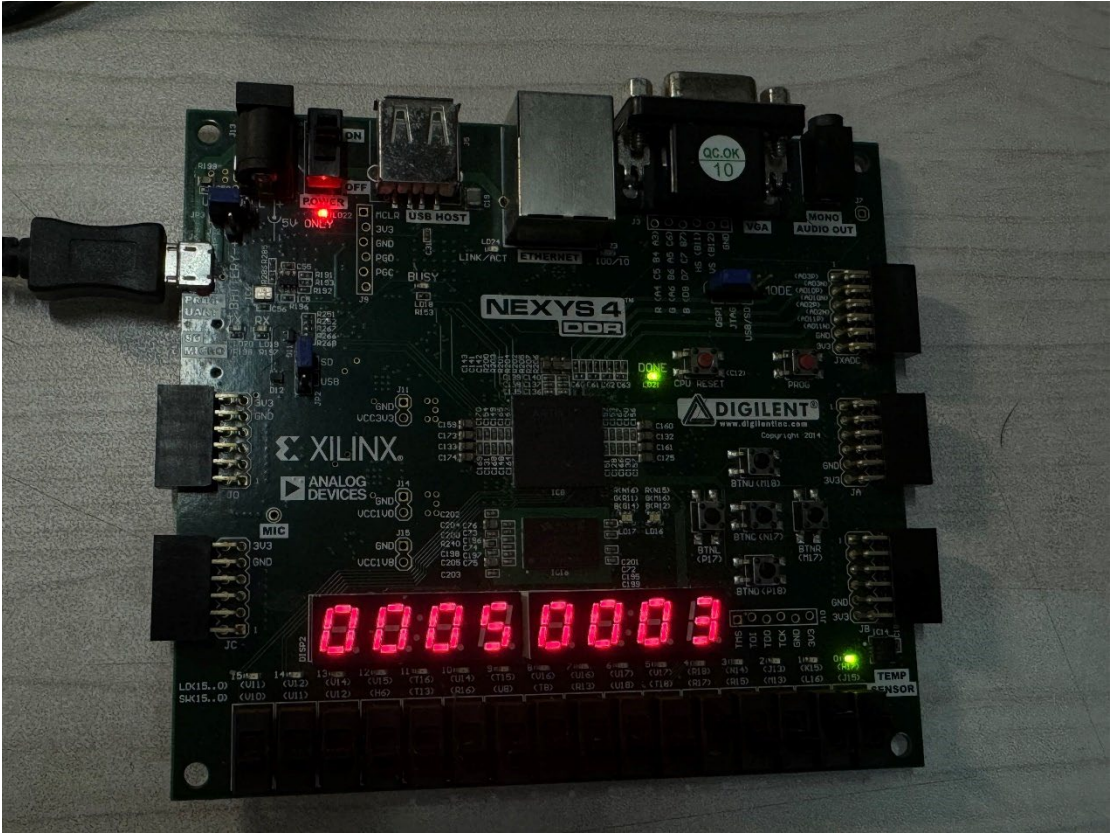


下面介绍下板实验的测试流程。上图下方黄色框线内的 16 个开关（对应 V10~J15）表示 16 位的数据输入，可能是初始输入的楼层高度或鸡蛋耐摔值。按钮 P17 与 M17 在高电平时，分别对应将楼层高度值与鸡蛋耐摔值送寄存。按钮 N17 表示复位键，此时将寄存的楼层高度值与鸡蛋耐摔值打入寄存器堆的对应寄存器（\$2 与 \$3）。计算出的尝试次数与摔破鸡蛋个数分别显示在七段数码管的左

右两端，LED 灯 H17 用于表示最后一个鸡蛋是否摔破。

3. 样例测试

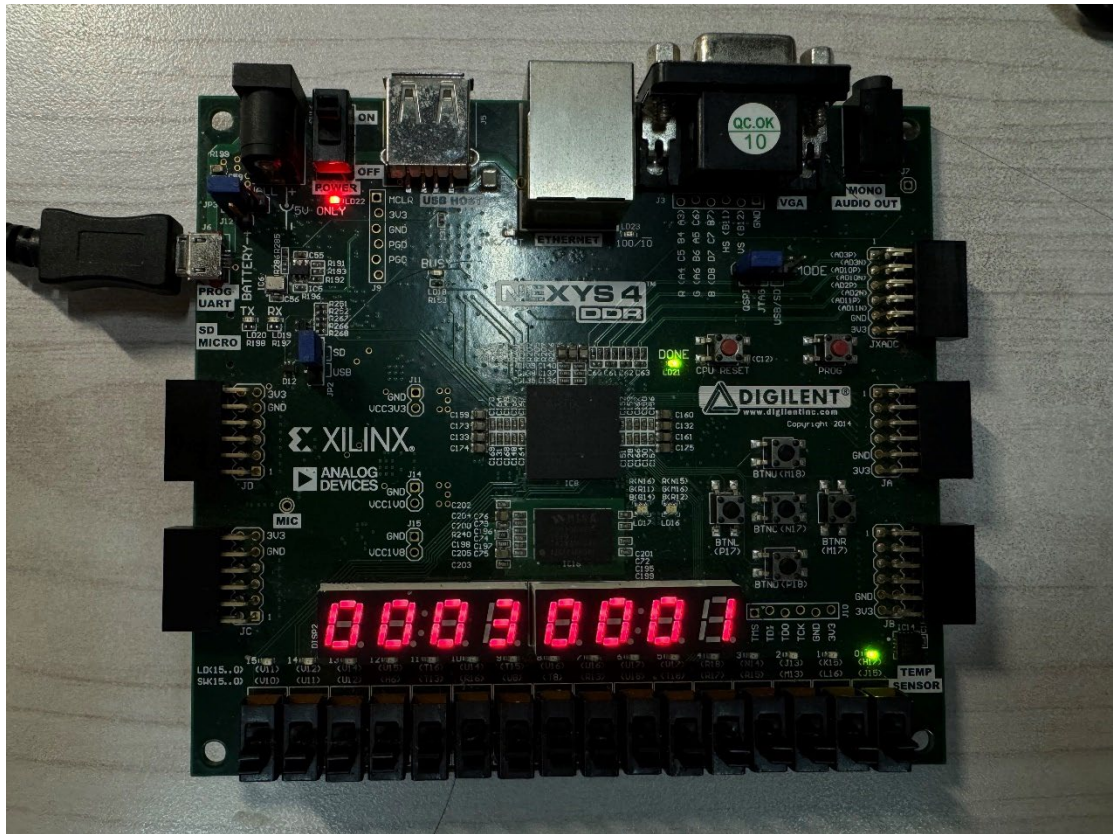
① 层高为 30，耐摔值为 19:



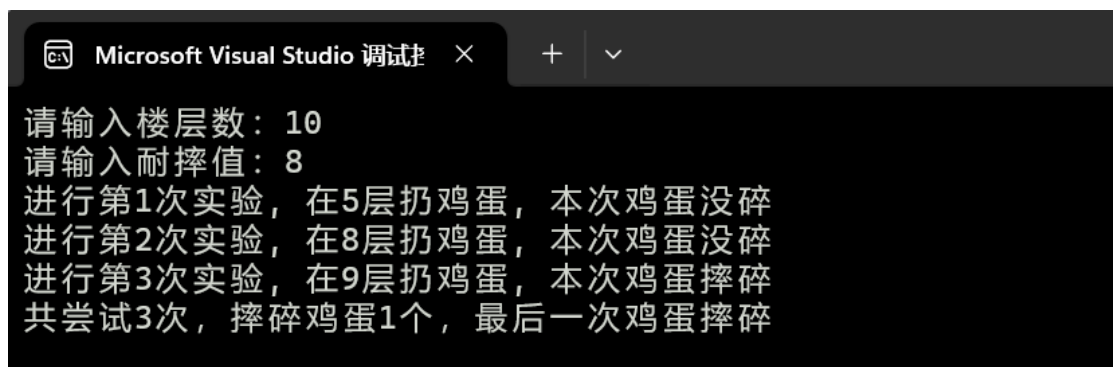
根据显示结果可得，测试次数为 5，摔坏鸡蛋个数为 3，最后摔坏了。与 C 语言程序相比较可发现结果正确：

```
Microsoft Visual Studio 调试 × + v
请输入楼层数：30
请输入耐摔值：19
进行第1次实验，在15层扔鸡蛋，本次鸡蛋没碎
进行第2次实验，在23层扔鸡蛋，本次鸡蛋摔碎
进行第3次实验，在19层扔鸡蛋，本次鸡蛋没碎
进行第4次实验，在21层扔鸡蛋，本次鸡蛋摔碎
进行第5次实验，在20层扔鸡蛋，本次鸡蛋摔碎
共尝试5次，摔碎鸡蛋3个，最后一次鸡蛋摔碎
```

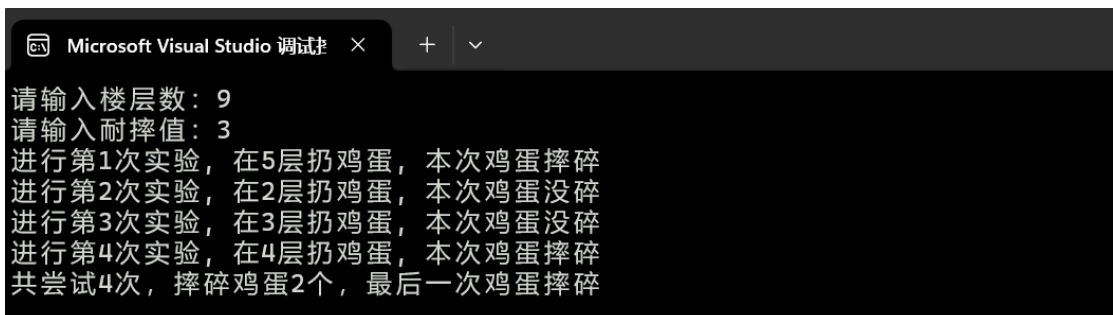
② 层高为 10，耐摔值为 8:



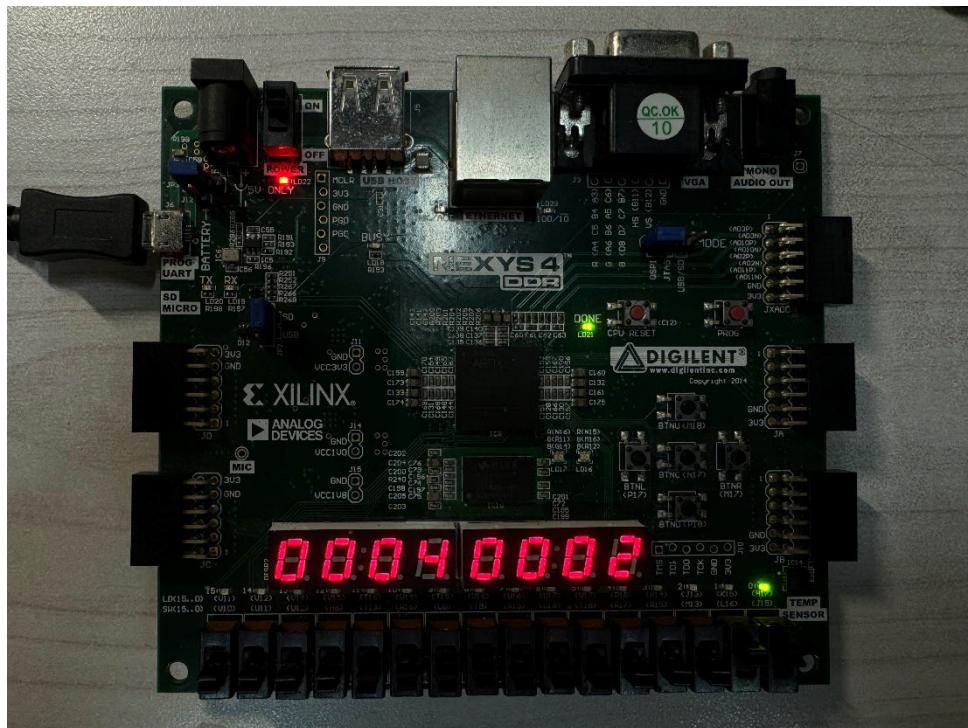
根据显示结果可得，测试次数为 3，摔坏鸡蛋个数为 1，最后摔坏了。与 C 语言程序相比较可发现结果正确：



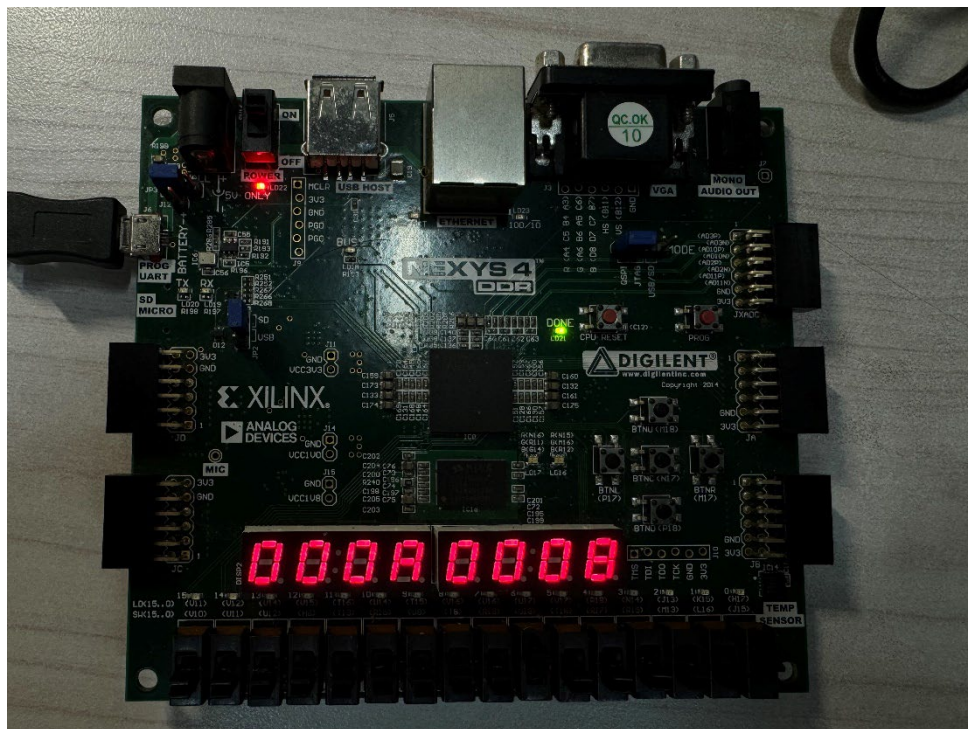
③ 层高为 9，耐摔值为 3：



根据显示结果可得，测试次数为 4，摔坏鸡蛋个数为 2，最后摔坏了。与 C 语言程序相比较可发现结果正确：



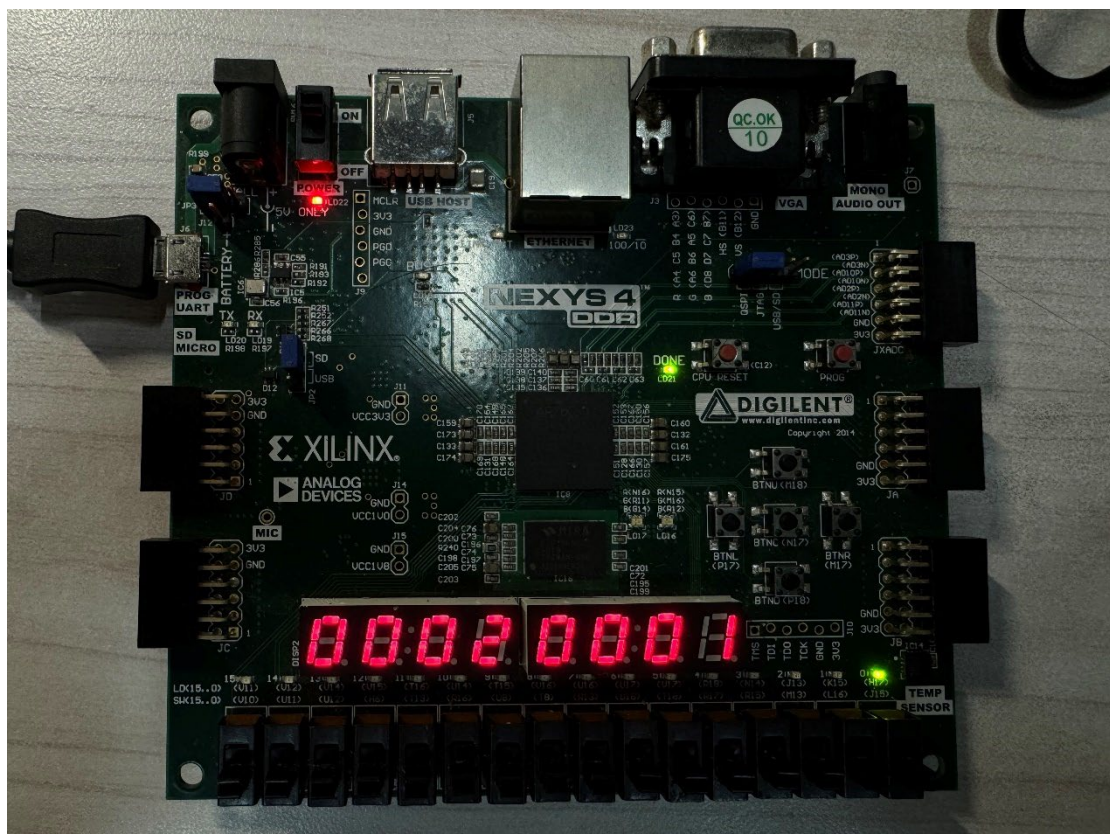
④ 层高为 1024，耐摔值为 65：



根据显示结果可得，测试次数为 10，摔坏鸡蛋个数为 8，最后没有摔坏。与 C 语言程序相比较可发现结果正确：

```
Microsoft Visual Studio 调试 × + ∨
请输入楼层数：1024
请输入耐摔值：65
进行第1次实验，在512层扔鸡蛋，本次鸡蛋摔碎
进行第2次实验，在256层扔鸡蛋，本次鸡蛋摔碎
进行第3次实验，在128层扔鸡蛋，本次鸡蛋摔碎
进行第4次实验，在64层扔鸡蛋，本次鸡蛋没碎
进行第5次实验，在96层扔鸡蛋，本次鸡蛋摔碎
进行第6次实验，在80层扔鸡蛋，本次鸡蛋摔碎
进行第7次实验，在72层扔鸡蛋，本次鸡蛋摔碎
进行第8次实验，在68层扔鸡蛋，本次鸡蛋摔碎
进行第9次实验，在66层扔鸡蛋，本次鸡蛋摔碎
进行第10次实验，在65层扔鸡蛋，本次鸡蛋没碎
共尝试10次，摔碎鸡蛋8个，最后一次鸡蛋没碎
```

⑤ 层高为 2，耐摔值为 1：



根据显示结果可得，测试次数为 2，摔坏鸡蛋个数为 1，最后摔坏了。与 C 语言程序相比较可发现结果正确：

```
Microsoft Visual Studio 调试 × + v
请输入楼层数：2
请输入耐摔值：1
进行第1次实验，在1层扔鸡蛋，本次鸡蛋没碎
进行第2次实验，在2层扔鸡蛋，本次鸡蛋摔碎
共尝试2次，摔碎鸡蛋1个，最后一次鸡蛋摔碎
```

八、流水线的性能指标定性分析（包括：吞吐率、加速比、效率及相关与冲突分析、CPU 的运行时间及存储器空间的使用）

由于比萨斜塔摔鸡蛋的测试程序中不包含访存指令，为了更好地分析流水线性能指标，我选取了两段不同的测试程序。第一个测试程序即以楼层数为 1024、鸡蛋耐摔值为 65 的情况为例；第二个测试程序是如下的冒泡排序：

```
.data
    Array: .space 40
.text
# 初始化数列
addi $2, $0, 5
addi $3, $0, 4
addi $4, $0, 3
addi $5, $0, 2
addi $6, $0, 1

# 将寄存器中的数值放入内存中以排序
addi $t0, $0, 0
sw $2, Array($t0)
    addi $t0, $t0, 4
sw $3, Array($t0)
    addi $t0, $t0, 4
sw $4, Array($t0)
    addi $t0, $t0, 4
sw $5, Array($t0)
    addi $t0, $t0, 4
sw $6, Array($t0)
```



```

    addi    $t1, $0, 0 # 控制外层循环

loop1:
    addi    $t2, $0, 0 # 控制内层循环

loop2:
    addi    $t3, $t2, 4    # 指向下一个数字
    lw     $t4, Array($t2)
    lw     $t5, Array($t3)
    slt    $t0, $t4, $t5    # 比较两数字，若小于则不交换
    bne    $t0, $zero, endl2
    sw     $t5, Array($t2) # 交换数字
    sw     $t4, Array($t3)

endl2:
    addi    $t2, $t2, 4
    sll    $t3, $t1, 2      # t1计数x4
    add    $t3, $t2, $t3
    beq    $t3, 16, endl1    # 比较循环结束
    j      loop2

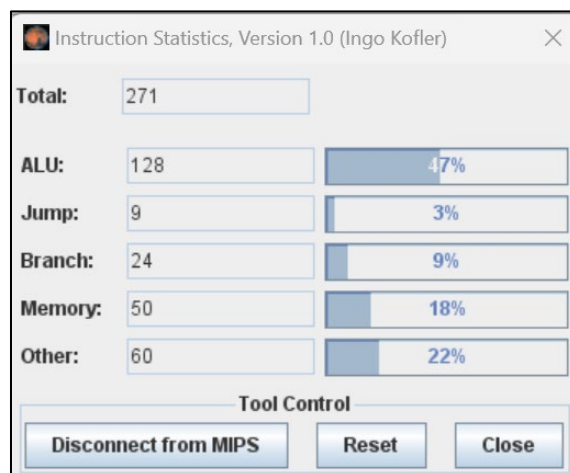
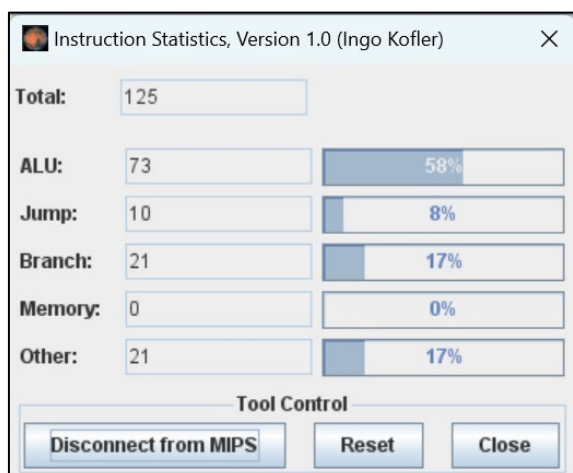
endl1:
    addi    $t1, $t1, 1    # 计数
    beq    $t1, 4, done
    j      loop1

done:
    # 将内存中排好序的数组移回寄存器中
    addi    $t0, $0, 0
    lw     $2, Array($t0)
        addi    $t0, $t0, 4
    lw     $3, Array($t0)
        addi    $t0, $t0, 4
    lw     $4, Array($t0)
        addi    $t0, $t0, 4
    lw     $5, Array($t0)
        addi    $t0, $t0, 4
    lw     $6, Array($t0)

end:

```

通过 MARS 分析，两个测试程序的指令条数分别如下（左图与右图）：



1. 吞吐率分析

① 测试程序 1

总共执行的指令数为 125 条，根据波形图分析得计算所用的总周期数是 213：

$$TP_1 = \frac{\text{指令数 } n}{\text{时钟周期数 } T_K} = \frac{125}{213} = 0.5869$$

② 测试程序 2

总共执行的指令数为 271 条，根据波形图分析得计算所用的总周期数是 502：

$$TP_2 = \frac{\text{指令数 } n}{\text{时钟周期数 } T_K} = \frac{271}{502} = 0.5398$$

2. 加速比分析

① 测试程序 1

125 条指令中，共有 10 条跳转指令、21 条分支指令（仅需要 IF、ID 两阶段），其余 94 条为运算指令（需要 IF、ID、EX、WB 四阶段）：

$$S_1 = \frac{\text{顺序执行时钟周期数 } T_S}{\text{流水线执行时钟周期数 } T_k} = \frac{94 \times 4 + 31 \times 2 \text{ 个时钟周期}}{213 \text{ 个时钟周期}} = 2.05$$

② 测试程序 2

271 条指令中，共有 9 条跳转指令、24 条分支指令（仅需要 IF、ID 两阶段），188 条为运算指令（需要 IF、ID、EX、WB 四阶段），50 条为访存指令（需要 IF、ID、EX、MEM、WB 五阶段），计算如下：

$$S_2 = \frac{\text{顺序执行时钟周期数 } T_S}{\text{流水线执行时钟周期数 } T_k} = \frac{50 \times 5 + 188 \times 4 + 33 \times 2 \text{ 个时钟周期}}{502 \text{ 个时钟周期}} = 2.13$$

3. 效率分析

本次流水线共分为 5 段，因此效率计算如下：

$$E_1 = \frac{S}{k} = \frac{104 \times 4 + 19 \times 2}{213 \times 5} = 41.13\%$$

$$E_2 = \frac{S}{k} = \frac{50 \times 5 + 188 \times 4 + 33 \times 2}{502 \times 5} = 42.55\%$$

4. 相关与冲突分析

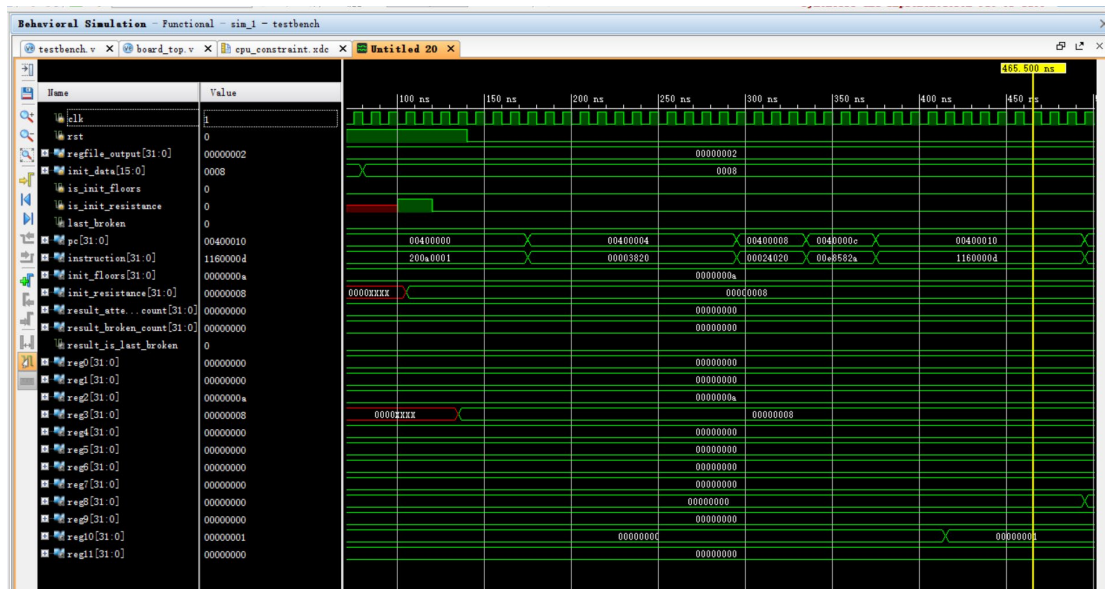
① 数据相关

采用暂停流水线（在流水线中插入流水线气泡）的方法来解决数据相关问题。读寄存器出现在 ID 段，写寄存器出现在 WB 段，且由于构造的 CPU 为静态流水线，所以只会出现先写后读（Read After Write, RAW）的数据相关。由于 WB 写回在上半个时钟周期，而 ID 读寄存器值在下半个时钟周期，因此 ID-WB 不存在 RAW 冲突。综上所述，需要检测的数据冲突为 ID-EX、ID-MEM 两种情况。

对于前者，需要插入两个“气泡”，等待当前 EX 数据执行到 WB 执行写入寄存器；对于后者，需要插入一个“气泡”，等待下一周期 MEM 阶段传入到 WB 阶段写入寄存器后，先写后读冲突解决。

Text Segment				
Bkpt	Address	Code	Basic	Source
	0x00400000	0x200a0001	addi \$10,\$0,0x00000001	17: addi \$10, \$0, 1 # 常数 1 (\$10=1)
	0x00400004	0x00003820	add \$7,\$0,\$0	18: add \$7, \$0, \$0 # 左边界 l=0
	0x00400008	0x00024020	add \$8,\$0,\$2	19: add \$8, \$0, \$2 # 右边界 r=n
	0x0040000c	0x00e8582a	slt \$11,\$7,\$8	22: slt \$11, \$7, \$8 # 检查左边界是否大于等于右边界 (\$11=0)
	0x00400010	0x1160000d	beq \$11,\$0,0x0000000d	23: beq \$11, \$0, EMD # 如果左边界大于等于右边界, 跳转到 EMD (
	0x00400014	0x20840001	addi \$4,\$4,0x00000001	24: addi \$4, \$4, 1 # 增加总拌鸡蛋次数
	0x00400018	0x00e84820	add \$9,\$7,\$8	25: add \$9, \$7, \$8
	0x0040001c	0x21290001	addi \$9,\$9,0x00000001	26: addi \$9, \$9, 1
	0x00400020	0x00094843	sra \$9,\$9,0x00000001	27: sra \$9, \$9, 1 # 计算中间值 mid = (l+r+1)/2
	0x00400024	0x0069582a	slt \$11,\$3,\$9	28: slt \$11, \$3, \$9 # 检查鸡蛋耐摔值是否大于等于中间值 (\$11=
	0x00400028	0x116a0003	beq \$11,\$10,0x00000003	29: beq \$11, \$10, BROKE # 如果鸡蛋耐摔值大于等于中间值, 跳转到 B
	0x0040002c	0x00003020	add \$6,\$0,\$0	31: add \$6, \$0, \$0 # 设置最后一个鸡蛋是否碎裂为 0
	0x00400030	0x00093820	add \$7,\$0,\$9	32: add \$7, \$0, \$9 # 更新左边界为中间值

由于数据相关而插入“气泡”如下图所示。当指令 0x400008 运行到 ID 阶段时，指令 0x400000 运行到 IF 取指阶段。此时 0x400008 指令从源寄存器\$7，\$8取操作数，分别与前两条指令有 ID-EX 和 ID-MEM 冲突，因此需要 stall 两周期。



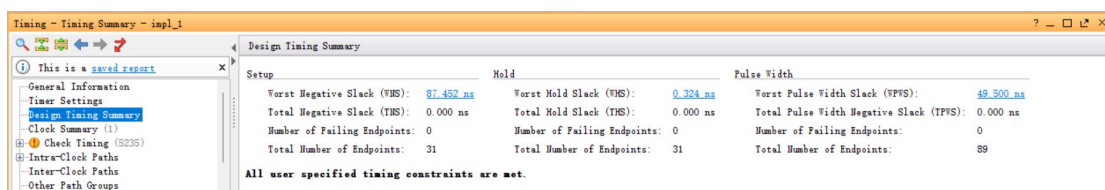
② 控制相关

对于分支和跳转指令(如 J、BEQ、BNE 等),采用冻结(freeze)或排空(flush)流水线的方式。当 ID 阶段检测到分支指令后,就暂停执行其后的指令,将计算得到的新 PC 值赋予 PC,再接着启动流水线。由于提前到 ID 译码段即进行分支判断,因此使得分支延迟为一个周期。

5. CPU 运行时间

测试程序的时间复杂度与楼高相关。若楼高为 n,则时间复杂度为 $O(\log n)$ 。

时序报告如下：



6. 存储器空间使用

- regfile: 32 个 32 位寄存器， $32 \times 32 = 1\text{Kbit}$
- IMEM: 2048 个 32 位寄存器， $2048 \times 32 = 64\text{Kbit}$
- DMEM: 2048 个 32 位寄存器， $2048 \times 32 = 64\text{Kbit}$

由于本测试程序中未使用数据存储器，因此实际上只用到了寄存器与指令存储器的部分存储空间。

九、总结与体会

在本次实验过程中，我自己设计并实现了基于 MIPS 指令集的五段静态流水线的 CPU。这个 CPU 的设计涵盖了指令的取指（IF）、译码（ID）、执行（EX）、访存（MEM）和写回（WB）等五个关键阶段，通过合理的控制信号设计和硬件优化，实现了指令的并行执行，提高了整体的运行效率。接下来，我将详细描述我的验证程序、在实验板上的验证过程以及性能分析的相关内容。

为了验证 CPU 的正确性和功能完整性，我编写了一套计算比萨斜塔摔鸡蛋这一实际问题的验证程序，其中包括了常见的 MIPS 指令，覆盖了 R 型、I 型、J 型不同类型的操作，涵盖算数指令、逻辑指令、分支指令等。这些测试用例旨在检验流水线各个阶段的正确执行，以及对数据冒险和控制冒险的处理是否得当。通过在 Vivado、Modelsim 上仿真运行这些测试用例，我能够确保 CPU 在各种情况下都能正确运行，并且能够及时发现和解决潜在的问题。

随后，我将验证程序加载到开发板上进行验证。在这个过程中，我需要确保硬件资源的正确约束配置和时钟频率的合理设定。通过实际的硬件验证，我可以更加直观地观察到流水线的运行情况，包括各个阶段的信号波形、数据通路的正

确连接以及时序的准确性。这一步骤是整个实验的关键，因为实验板上的验证结果更接近实际应用场景，可以更全面地检验 CPU 的性能。

在验证程序正确运行后，我进行了性能分析，主要关注了吞吐率、加速比和效率等方面。吞吐率是指单位时间内完成的指令数量，加速比是将流水线与单周期设计进行比较的一个重要指标，效率则衡量了流水线中设备实际使用时间与整个运行时间的比值。通过这些性能指标的分析，我能够客观地评估流水线设计的优劣，验证其在提高指令执行效率上的有效性。此外，我在验证过程中也注意到一些问题和改进的空间。例如，在处理某些特殊指令或者特定的数据冒险情况时，流水线的性能可能存在一些瓶颈，需要进一步优化。这种对性能瓶颈的识别和改进是实际工程中非常重要的一步，可以为流水线的后续优化提供有力的参考。

综上所述，通过验证程序、实验板上的验证和性能分析，我不仅验证了 CPU 的正确性和功能完整性，也全面评估了流水线的性能。这一过程不仅提高了我对计算机体系结构的理解，也培养了我硬件设计和性能优化方面的实际应用能力。希望能在本学期的计算机系统结构实验过程中锻炼自己的硬件开发能力，更好的地将课内学习到的知识亲自实践，不断进步！

十、附件（所有程序）

1. board_top.v

```
`timescale 1ns / 1ns

module board_top(
    input          in_clk,
    input          in_rst,

    input  [15:0]  in_data,
    input          is_init_floors,
    input          is_init_resistance,

    output [7:0]   o_seg,
    output [7:0]   o_sel,

    output          result_is_last_broken
);

    wire  [31:0]  pc;
    wire  [31:0]  instruction;

    reg  [15:0]   init_floors;
    reg  [15:0]   init_resistance;

    wire  [31:0]  result_attempt_count;
    wire  [31:0]  result_broken_count;
    wire          new_clk;

    clk_divider #(4) clk_divider_uut(
        .in_clk(in_clk),
        .out_clk(new_clk)
    );

    always @(posedge in_clk)
    begin
        // 初始化楼层数
        if(is_init_floors)
            init_floors    <= in_data;
        // 初始化耐摔值
        else if(is_init_resistance)
            init_resistance <= in_data;
    end
endmodule
```

```

end

cpu cpu_uut(
    .in_clk(new_clk),
    .in_rst(in_rst),

    .init_floors({ 16'b0, init_floors }),
    .init_resistance({ 16'b0, init_resistance }),

    .out_pc(pc),
    .out_instruction(instruction),

    .result_attempt_count(result_attempt_count),
    .result_broken_count(result_broken_count),
    .result_is_last_broken(result_is_last_broken)
);

seg7x16 seg7(
    .clk(in_clk),
    .reset(in_rst),
    .cs(1'b1),
    .i_data({ result_attempt_count[15:0],
result_broken_count[15:0] }),
    .o_seg(o_seg),
    .o_sel(o_sel)
);

endmodule

```

2. cpu.v

```

`timescale 1ns / 1ns

module cpu(
    input          in_clk,
    input          in_rst,

    input  [31:0]  init_floors,
    input  [31:0]  init_resistance,

    output [31:0]  out_pc,

```

```
output [31:0] out_instruction,

output [31:0] result_attempt_count,
output [31:0] result_broken_count,
output      result_is_last_broken
);
```

```
/* IF段 */
```

```
wire [31:0] if_out_pc;
wire [31:0] if_out_npc;
wire [31:0] if_out_instruction;
```

```
/* ID段 */
```

```
wire [31:0] id_in_npc;
wire [31:0] id_in_instruction;
wire [4:0]  id_in_ex_waddr;
wire [4:0]  id_in_mem_waddr;
wire       id_in_wena;
wire       id_in_ex_wena;
wire       id_in_mem_wena;
```

```
wire       id_out_dmem_ena;
wire       id_out_dmem_wena;
wire [1:0] id_out_dmem_type;
wire [31:0] id_out_rs_data;
wire [31:0] id_out_rt_data;
wire [4:0]  id_out_rd_waddr;
wire       id_out_rd_sel;
wire       id_out_rd_wena;
wire [31:0] id_out_immed;
wire [31:0] id_out_shamt;
wire [1:0]  id_out_pc_sel;
wire [31:0] id_out_pc_baddr;
wire [31:0] id_out_pc_jaddr;
wire       id_out_alu_a_sel;
wire       id_out_alu_b_sel;
wire [3:0]  id_out_alu_sel;
wire       id_out_stall;
wire       id_out_branch;
```

```
/* EX段 */
```

```
wire       ex_in_dmem_ena;
```

```

wire          ex_in_dmem_wena;
wire [1:0]    ex_in_dmem_type;
wire [31:0]   ex_in_rs_data;
wire [31:0]   ex_in_rt_data;
wire [4:0]    ex_in_rd_waddr;
wire          ex_in_rd_wena;
wire          ex_in_rd_sel;
wire [31:0]   ex_in_immed;
wire [31:0]   ex_in_shamt;
wire          ex_in_alu_a_sel;
wire          ex_in_alu_b_sel;
wire [3:0]    ex_in_alu_sel;
wire          ex_in_stall;

wire          ex_out_dmem_ena;
wire          ex_out_dmem_wena;
wire [1:0]    ex_out_dmem_type;
wire [31:0]   ex_out_rs_data;
wire [31:0]   ex_out_rt_data;
wire [4:0]    ex_out_rd_waddr;
wire          ex_out_rd_sel;
wire          ex_out_rd_wena;
wire [31:0]   ex_out_alu_result;

/* MEM段 */
wire          mem_in_dmem_ena;
wire          mem_in_dmem_wena;
wire [1:0]    mem_in_dmem_type;
wire [31:0]   mem_in_rs_data;
wire [31:0]   mem_in_rt_data;
wire [4:0]    mem_in_rd_waddr;
wire          mem_in_rd_sel;
wire          mem_in_rd_wena;
wire [31:0]   mem_in_alu_result;

wire [4:0]    mem_out_rd_waddr;
wire          mem_out_rd_sel;
wire          mem_out_rd_wena;
wire [31:0]   mem_out_alu_result;
wire [31:0]   mem_out_dmem_data;

/* WB段 */
wire [4:0]    wb_in_rd_waddr;

```



```

wire          wb_in_rd_sel;
wire          wb_in_rd_wena;
wire [31:0]   wb_in_alu_result;
wire [31:0]   wb_in_dmem_data;

wire [4:0]    wb_out_rd_waddr;
wire          wb_out_rd_wena;
wire [31:0]   wb_out_rd_data;

assign out_pc          = if_out_pc;
assign out_instruction = if_out_instruction;

pipe_if pipe_if_uut(
    .in_clk(in_clk),
    .in_rst(in_rst),
    .in_stall(id_out_stall),

    .in_pc_baddr(id_out_pc_baddr),
    .in_pc_jaddr(id_out_pc_jaddr),
    .in_pc_sel(id_out_pc_sel),

    .out_pc(if_out_pc),
    .out_npc(if_out_npc),
    .out_instruction(if_out_instruction)
);

pipe_if_id pipe_if_id_uut(
    .in_clk(in_clk),
    .in_rst(in_rst),

    .in_npc(if_out_npc),
    .in_instruction(if_out_instruction),

    .in_stall(id_out_stall),
    .in_branch(id_out_branch),

    .out_npc(id_in_npc),
    .out_instruction(id_in_instruction)
);

pipe_id pipe_id_uut(
    .in_clk(in_clk),
    .in_rst(in_rst),

```

```

.in_npc(id_in_npc),
.in_instruction(id_in_instruction),

.in_ex_waddr(ex_out_rd_waddr),
.in_mem_waddr(mem_out_rd_waddr),
.in_ex_wena(ex_out_rd_wena),
.in_mem_wena(mem_out_rd_wena),

.in_wb_reg_addr(wb_out_rd_waddr),
.in_wb_reg_data(wb_out_rd_data),
.in_wb_reg_ena(wb_out_rd_wena),

.init_floors(init_floors),
.init_resistance(init_resistance),

.out_rs_data(id_out_rs_data),
.out_rt_data(id_out_rt_data),
.out_rd_waddr(id_out_rd_waddr),
.out_rd_wena(id_out_rd_wena),
.out_rd_sel(id_out_rd_sel),
.out_immed(id_out_immed),
.out_shamt(id_out_shamt),

.out_dmem_ena(id_out_dmem_ena),
.out_dmem_wena(id_out_dmem_wena),
.out_dmem_type(id_out_dmem_type),

.out_alu_a_sel(id_out_alu_a_sel),
.out_alu_b_sel(id_out_alu_b_sel),
.out_alu_sel(id_out_alu_sel),

.out_pc_sel(id_out_pc_sel),
.out_pc_baddr(id_out_pc_baddr),
.out_pc_jaddr(id_out_pc_jaddr),

.out_stall(id_out_stall),
.out_branch(id_out_branch),

.result_attempt_count(result_attempt_count),
.result_broken_count(result_broken_count),
.result_is_last_broken(result_is_last_broken)
);

```

```

pipe_id_ex pipe_id_ex_uut(
    .in_clk(in_clk),
    .in_rst(in_rst),

    .in_dmem_ena(id_out_dmem_ena),
    .in_dmem_wena(id_out_dmem_wena),
    .in_dmem_type(id_out_dmem_type),

    .in_rs_data(id_out_rs_data),
    .in_rt_data(id_out_rt_data),

    .in_rd_waddr(id_out_rd_waddr),
    .in_rd_sel(id_out_rd_sel),
    .in_rd_wena(id_out_rd_wena),

    .in_alu_a_sel(id_out_alu_a_sel),
    .in_alu_b_sel(id_out_alu_b_sel),
    .in_alu_sel(id_out_alu_sel),

    .in_immed(id_out_immed),
    .in_shamt(id_out_shamt),

    .in_stall(id_out_stall),

    .out_dmem_ena(ex_in_dmem_ena),
    .out_dmem_wena(ex_in_dmem_wena),
    .out_dmem_type(ex_in_dmem_type),

    .out_rs_data(ex_in_rs_data),
    .out_rt_data(ex_in_rt_data),
    .out_rd_waddr(ex_in_rd_waddr),
    .out_rd_sel(ex_in_rd_sel),
    .out_rd_wena(ex_in_rd_wena),

    .out_alu_a_sel(ex_in_alu_a_sel),
    .out_alu_b_sel(ex_in_alu_b_sel),
    .out_alu_sel(ex_in_alu_sel),

    .out_immed(ex_in_immed),
    .out_shamt(ex_in_shamt)
);

```

```

pipe_ex pipe_ex_uut(
    .in_rst(in_rst),

    .in_dmem_ena(ex_in_dmem_ena),
    .in_dmem_wena(ex_in_dmem_wena),
    .in_dmem_type(ex_in_dmem_type),

    .in_rs_data(ex_in_rs_data),
    .in_rt_data(ex_in_rt_data),
    .in_rd_waddr(ex_in_rd_waddr),
    .in_rd_sel(ex_in_rd_sel),
    .in_rd_wena(ex_in_rd_wena),

    .in_immed(ex_in_immed),
    .in_shamt(ex_in_shamt),

    .in_alu_a_sel(ex_in_alu_a_sel),
    .in_alu_b_sel(ex_in_alu_b_sel),
    .in_alu_sel(ex_in_alu_sel),

    .out_dmem_ena(ex_out_dmem_ena),
    .out_dmem_wena(ex_out_dmem_wena),
    .out_dmem_type(ex_out_dmem_type),

    .out_rs_data(ex_out_rs_data),
    .out_rt_data(ex_out_rt_data),
    .out_rd_waddr(ex_out_rd_waddr),
    .out_rd_sel(ex_out_rd_sel),
    .out_rd_wena(ex_out_rd_wena),

    .out_alu_result(ex_out_alu_result)
);

```

```

pipe_ex_mem pipe_ex_mem_uut(
    .in_clk(in_clk),
    .in_rst(in_rst),

    .in_dmem_ena(ex_out_dmem_ena),
    .in_dmem_wena(ex_out_dmem_wena),
    .in_dmem_type(ex_out_dmem_type),

    .in_rs_data(ex_out_rs_data),
    .in_rt_data(ex_out_rt_data),

```

```

        .in_rd_waddr(ex_out_rd_waddr),
        .in_rd_sel(ex_out_rd_sel),
        .in_rd_wena(ex_out_rd_wena),

        .in_alu_result(ex_out_alu_result),

        .out_dmem_ena(mem_in_dmem_ena),
        .out_dmem_wena(mem_in_dmem_wena),
        .out_dmem_type(mem_in_dmem_type),

        .out_rs_data(mem_in_rs_data),
        .out_rt_data(mem_in_rt_data),
        .out_rd_waddr(mem_in_rd_waddr),
        .out_rd_sel(mem_in_rd_sel),
        .out_rd_wena(mem_in_rd_wena),

        .out_alu_result(mem_in_alu_result)
    );

    pipe_mem pipe_mem_uut(
        .in_clk(in_clk),

        .in_dmem_ena(mem_in_dmem_ena),
        .in_dmem_wena(mem_in_dmem_wena),
        .in_dmem_type(mem_in_dmem_type),

        .in_rs_data(mem_in_rs_data),
        .in_rt_data(mem_in_rt_data),
        .in_rd_waddr(mem_in_rd_waddr),
        .in_rd_sel(mem_in_rd_sel),
        .in_rd_wena(mem_in_rd_wena),

        .in_alu_result(mem_in_alu_result),

        .out_rd_waddr(mem_out_rd_waddr),
        .out_rd_wena(mem_out_rd_wena),
        .out_rd_sel(mem_out_rd_sel),

        .out_alu_result(mem_out_alu_result),

        .out_dmem_data(mem_out_dmem_data)
    );

```

```

    pipe_mem_wb pipe_mem_wb_uut(
        .in_clk(in_clk),
        .in_rst(in_rst),

        .in_rd_waddr(mem_out_rd_waddr),
        .in_rd_sel(mem_out_rd_sel),
        .in_rd_wena(mem_out_rd_wena),

        .in_alu_result(mem_out_alu_result),
        .in_dmem_data(mem_out_dmem_data),

        .out_rd_waddr(wb_in_rd_waddr),
        .out_rd_sel(wb_in_rd_sel),
        .out_rd_wena(wb_in_rd_wena),

        .out_alu_result(wb_in_alu_result),
        .out_dmem_data(wb_in_dmem_data)
    );

    pipe_wb pipe_wb_uut(
        .in_rd_waddr(wb_in_rd_waddr),
        .in_rd_sel(wb_in_rd_sel),
        .in_rd_wena(wb_in_rd_wena),

        .in_alu_result(wb_in_alu_result),
        .in_dmem_data(wb_in_dmem_data),

        .out_rd_waddr(wb_out_rd_waddr),
        .out_rd_wdata(wb_out_rd_data),
        .out_rd_wena(wb_out_rd_wena)
    );

endmodule

```

3. pipeline_if.v

```

`timescale 1ns / 1ns

module pipe_if(
    input          in_clk,

```

```

input          in_rst,
input          in_stall,
input  [31:0]  in_pc_jaddr,
input  [31:0]  in_pc_baddr,
input  [1:0]   in_pc_sel,
output [31:0]  out_pc,
output [31:0]  out_npc,
output [31:0]  out_instruction
);

wire [31:0] next_pc;

pcreg pcreg_uut(
    .clk(in_clk),
    .ena(1'b1),
    .rst(in_rst),
    .stall(in_stall),
    .pc_in(next_pc),
    .pc_out(out_pc)
);

mux4_32 mux4_pc_sel_uut(
    .d0(out_npc),
    .d1(in_pc_baddr),
    .d2(in_pc_jaddr),
    .d3(32'bz),
    .y(next_pc),
    .s(in_pc_sel)
);

assign out_npc = out_pc + 4;

imem imem_uut(out_pc >> 2, out_instruction);

endmodule

```

4. pipeline_if_id.v

```

`timescale 1ns / 1ns

```

```

module pipe_if_id(
    input          in_clk,
    input          in_rst,
    input          in_stall,
    input          in_branch,
    input  [31:0]  in_npc,
    input  [31:0]  in_instruction,
    output reg  [31:0]  out_npc,
    output reg  [31:0]  out_instruction
);

always @(posedge in_clk or posedge in_rst)
begin
    if(in_rst)
    begin
        out_npc          <= 32'b0;
        out_instruction <= 32'b0;
    end
    else if(~in_stall)
    begin
        if(in_branch)
        begin
            out_npc          <= 32'b0;
            out_instruction <= 32'b0;
        end
        else
        begin
            out_npc          <= in_npc;
            out_instruction <= in_instruction;
        end
    end
    else
    begin
        out_npc          <= out_npc;
        out_instruction <= out_instruction;
    end
end

endmodule

```

5. pipeline_id.v

```

`timescale 1ns / 1ns

```



```

module pipe_id(
    input          in_clk,
    input          in_rst,

    input  [31:0]  in_npc,
    input  [31:0]  in_instruction,

    input  [4:0]   in_ex_waddr,
    input  [4:0]   in_mem_waddr,
    input          in_ex_wena,
    input          in_mem_wena,

    input  [4:0]   in_wb_reg_addr,
    input          in_wb_reg_ena,
    input  [31:0]  in_wb_reg_data,

    input  [31:0]  init_floors,
    input  [31:0]  init_resistance,

    output [31:0]   out_rs_data,
    output [31:0]   out_rt_data,
    output [4:0]    out_rd_waddr,
    output          out_rd_sel,
    output          out_rd_wena,
    output [31:0]   out_immed,
    output [31:0]   out_shamt,

    output          out_dmem_ena,
    output          out_dmem_wena,
    output [1:0]    out_dmem_type,

    output [31:0]   out_pc_baddr,
    output [31:0]   out_pc_jaddr,
    output [1:0]    out_pc_sel,

    output          out_alu_a_sel,
    output          out_alu_b_sel,
    output [3:0]    out_alu_sel,

    output          out_stall,
    output          out_branch,

```

```

output [31:0]      result_attempt_count,
output [31:0]      result_broken_count,
output            result_is_last_broken
);

// 解析指令的各个部分
wire [5:0] inst_op   = in_instruction[31:26];
wire [5:0] inst_func = in_instruction[ 5: 0];
wire [4:0] rs_addr   = in_instruction[25:21];
wire [4:0] rt_addr   = in_instruction[20:16];
wire [4:0] rd_addr   = in_instruction[15:11];

wire rs_rena, rt_rena;
wire ext_signed;

assign out_immed = { { 16{ ext_signed &
in_instruction[15] } }, in_instruction[15:0] };
assign out_shamt = { 27'b0, in_instruction[10:6] };

assign out_pc_baddr = in_npc +
{ { 14{ in_instruction[15] } }, in_instruction[15:0], 2'b0 };
assign out_pc_jaddr = { in_npc[31:28],
in_instruction[25:0], 2'b0 };
assign out_npc = in_npc;

assign out_branch = (((inst_op == 6'b000100) &&
(out_rs_data == out_rt_data)) || ((inst_op == 6'b000101) &&
(out_rs_data != out_rt_data)) || (inst_op == 6'b000010));

// 寄存器堆
regfile regfile_uut(
    .in_clk(in_clk),
    .in_rst(in_rst),

    .in_rs_rena(rs_rena),
    .in_rt_rena(rt_rena),
    .in_rd_wena(in_wb_reg_ena),
    .in_rs_addr(rs_addr),
    .in_rt_addr(rt_addr),
    .in_rd_addr(in_wb_reg_addr),
    .in_rd_data(in_wb_reg_data),

    .init_floors(init_floors),

```

```

        .init_resistance(init_resistance),

        .out_rs_data(out_rs_data),
        .out_rt_data(out_rt_data),

        .result_attempt_count(result_attempt_count),
        .result_broken_count(result_broken_count),
        .result_is_last_broken(result_is_last_broken)
    );

```

// 控制器 解析指令，发出控制信号

```

controller controller_uut(
    .in_branch(out_branch),
    .in_instruction(in_instruction),

    .out_rs_rena(rs_rena),
    .out_rt_rena(rt_rena),
    .out_rd_wena(out_rd_wena),
    .out_rd_sel(out_rd_sel),
    .out_rd_addr(out_rd_waddr),

    .out_dmem_ena(out_dmem_ena),
    .out_dmem_wena(out_dmem_wena),
    .out_dmem_type(out_dmem_type),

    .out_ext_signed(ext_signed),
    .out_alu_a_sel(out_alu_a_sel),
    .out_alu_b_sel(out_alu_b_sel),
    .out_alu_sel(out_alu_sel),
    .out_pc_sel(out_pc_sel)
);

```

// 判断数据冲突，若有则发 stall 信号

```

stall stall_uut(
    .in_clk(in_clk),
    .in_rst(in_rst),
    .in_rs_addr(rs_addr),
    .in_rt_addr(rt_addr),
    .in_rs_rena(rs_rena),
    .in_rt_rena(rt_rena),
    .in_ex_waddr(in_ex_waddr),
    .in_mem_waddr(in_mem_waddr),
    .in_ex_wena(in_ex_wena),

```

```

        .in_mem_wena(in_mem_wena),
        .out_stall(out_stall)
    );

endmodule

```

6. pipeline_id_ex.v

```

`timescale 1ns / 1ns

module pipe_id_ex(
    input                in_clk,
    input                in_rst,

    input                in_dmem_ena,
    input                in_dmem_wena,
    input                [1:0] in_dmem_type,

    input                [31:0] in_rs_data,
    input                [31:0] in_rt_data,
    input                [4:0] in_rd_waddr,
    input                in_rd_sel,
    input                in_rd_wena,

    input                [31:0] in_immed,
    input                [31:0] in_shamt,

    input                in_alu_a_sel,
    input                in_alu_b_sel,
    input                [3:0] in_alu_sel,

    input                in_stall,

    output reg           out_dmem_ena,
    output reg           out_dmem_wena,
    output reg           [1:0] out_dmem_type,

    output reg           [31:0] out_rs_data,
    output reg           [31:0] out_rt_data,
    output reg           [4:0] out_rd_waddr,
    output reg           out_rd_sel,

```

```

output reg          out_rd_wena,

output reg  [31:0]  out_immed,
output reg  [31:0]  out_shamt,

output reg          out_alu_a_sel,
output reg          out_alu_b_sel,
output reg  [3:0]   out_alu_sel
);

always@ (posedge in_clk or posedge in_rst)
begin
    // 流水线排空
    if(in_rst || in_stall)
    begin
        out_dmem_ena    <= 1'b0;
        out_dmem_wena   <= 1'b0;
        out_dmem_type   <= 2'b0;

        out_rs_data     <= 32'b0;
        out_rt_data     <= 32'b0;
        out_rd_waddr     <= 5'b0;
        out_rd_sel       <= 1'b0;
        out_rd_wena      <= 1'b0;

        out_immed        <= 32'b0;
        out_shamt        <= 32'b0;

        out_alu_a_sel    <= 1'b0;
        out_alu_b_sel    <= 1'b0;
        out_alu_sel      <= 4'b0;
    end
    else
    begin
        out_dmem_ena     <= in_dmem_ena;
        out_dmem_wena    <= in_dmem_wena;
        out_dmem_type    <= in_dmem_type;

        out_rs_data      <= in_rs_data;
        out_rt_data      <= in_rt_data;
        out_rd_waddr     <= in_rd_waddr;
        out_rd_sel       <= in_rd_sel;
        out_rd_wena      <= in_rd_wena;
    end
end

```

```

        out_immed      <= in_immed;
        out_shamt      <= in_shamt;

        out_alu_a_sel  <= in_alu_a_sel;
        out_alu_b_sel  <= in_alu_b_sel;
        out_alu_sel    <= in_alu_sel;

    end
end
endmodule

```

7. pipeline_ex.v

```

`timescale 1ns / 1ns

module pipe_ex(
    input          in_rst,

    input          in_dmem_ena,
    input          in_dmem_wena,
    input  [1:0]   in_dmem_type,

    input  [31:0]  in_rs_data,
    input  [31:0]  in_rt_data,
    input  [4:0]   in_rd_waddr,
    input          in_rd_sel,
    input          in_rd_wena,

    input  [31:0]  in_immed,
    input  [31:0]  in_shamt,

    input          in_alu_a_sel,
    input          in_alu_b_sel,
    input  [3:0]   in_alu_sel,

    output          out_dmem_ena,
    output          out_dmem_wena,
    output  [1:0]   out_dmem_type,

    output  [31:0]  out_rs_data,

```

```

output  [31:0]  out_rt_data,
output  [4:0]   out_rd_waddr,
output                    out_rd_sel,
output                    out_rd_wena,

output  [31:0]  out_alu_result
);

wire [31:0] a;
wire [31:0] b;
wire zero, carry, negative, overflow;

assign out_dmem_ena      = in_dmem_ena;
assign out_dmem_wena     = in_dmem_wena;
assign out_dmem_type     = in_dmem_type;

assign out_rs_data       = in_rs_data;
assign out_rt_data       = in_rt_data;
assign out_rd_waddr      = in_rd_waddr;
assign out_rd_sel        = in_rd_sel;
assign out_rd_wena       = in_rd_wena;

assign a = in_alu_a_sel ? in_shamt : in_rs_data;
assign b = in_alu_b_sel ? in_immed : in_rt_data;

alu alu_uut(
    .a(a),
    .b(b),
    .y(out_alu_result),
    .aluc(in_alu_sel),
    .zero(zero),
    .carry(carry),
    .negative(negative),
    .overflow(overflow)
);

endmodule

```

8. pipeline_ex_mem.v

```

`timescale 1ns / 1ns

module pipe_ex_mem(
    input                in_clk,
    input                in_rst,

    input                in_dmem_ena,
    input                in_dmem_wena,
    input    [1:0]       in_dmem_type,

    input    [31:0]      in_rs_data,
    input    [31:0]      in_rt_data,
    input    [4:0]       in_rd_waddr,
    input                in_rd_sel,
    input                in_rd_wena,

    input    [31:0]      in_alu_result,

    output reg           out_dmem_ena,
    output reg           out_dmem_wena,
    output reg    [1:0]  out_dmem_type,

    output reg    [31:0] out_rs_data,
    output reg    [31:0] out_rt_data,
    output reg    [4:0]  out_rd_waddr,
    output reg           out_rd_sel,
    output reg           out_rd_wena,

    output reg    [31:0] out_alu_result
);

always@ (posedge in_clk or posedge in_rst)
begin
    if(in_rst)
    begin
        out_dmem_ena    <= 1'b0;
        out_dmem_wena    <= 1'b0;
        out_dmem_type    <= 2'b0;

        out_rs_data      <= 32'b0;
        out_rt_data      <= 32'b0;
        out_rd_waddr      <= 5'b0;
        out_rd_sel        <= 1'b0;
    end
end

```



```

        out_rd_wena      <= 1'b0;

        out_alu_result   <= 32'b0;
    end
    else
    begin
        out_dmem_ena     <= in_dmem_ena;
        out_dmem_wena    <= in_dmem_wena;
        out_dmem_type    <= in_dmem_type;

        out_rs_data      <= in_rs_data;
        out_rt_data      <= in_rt_data;
        out_rd_waddr     <= in_rd_waddr;
        out_rd_sel       <= in_rd_sel;
        out_rd_wena      <= in_rd_wena;

        out_alu_result   <= in_alu_result;
    end
end

endmodule

```

9. pipeline_mem.v

```

`timescale 1ns / 1ns

module pipe_mem(
    input          in_clk,

    input          in_dmem_ena,
    input          in_dmem_wena,
    input  [1:0]   in_dmem_type,

    input  [31:0]  in_rs_data,
    input  [31:0]  in_rt_data,
    input  [4:0]   in_rd_waddr,
    input          in_rd_sel,
    input          in_rd_wena,

    input  [31:0]  in_alu_result,

```

```

output [4:0] out_rd_waddr,
output      out_rd_sel,
output      out_rd_wena,

output [31:0] out_alu_result,
output [31:0] out_dmem_data
);

wire [31:0] dmem_addr = (in_alu_result - 32'h10010000) / 4;

assign out_rd_waddr = in_rd_waddr;
assign out_rd_sel   = in_rd_sel;
assign out_rd_wena  = in_rd_wena;

assign out_alu_result = in_alu_result;

dmem dmem_uut(
    .clk(in_clk),
    .ena(in_dmem_ena),
    .wena(in_dmem_wena),
    .addr(dmem_addr),
    .dmem_type(in_dmem_type),
    .data_in(in_rt_data),
    .data_out(out_dmem_data)
);

endmodule

```

10. pipeline_mem_wb.v

```

`timescale 1ns / 1ns

module pipe_mem_wb(
    input          in_clk,
    input          in_rst,

    input [4:0]    in_rd_waddr,
    input          in_rd_sel,
    input          in_rd_wena,

```

```

input      [31:0] in_alu_result,
input      [31:0] in_dmem_data,

output reg  [4:0]  out_rd_waddr,
output reg          out_rd_wena,
output reg          out_rd_sel,

output reg  [31:0] out_alu_result,
output reg  [31:0] out_dmem_data
);

always @(posedge in_clk or posedge in_rst)
begin
    if(in_rst == 1'b1)
    begin
        out_rd_waddr    <= 5'b0;
        out_rd_sel      <= 1'b0;
        out_rd_wena      <= 1'b0;

        out_alu_result  <= 32'b0;
        out_dmem_data   <= 32'b0;
    end
    else
    begin
        out_rd_waddr    <= in_rd_waddr;
        out_rd_sel      <= in_rd_sel;
        out_rd_wena      <= in_rd_wena;

        out_alu_result  <= in_alu_result;
        out_dmem_data   <= in_dmem_data;
    end
end
endmodule

```

11. pipeline_wb.v

```

`timescale 1ns / 1ns
module pipe_wb(
    input  [4:0]  in_rd_waddr,

```

```

input          in_rd_wena,
input          in_rd_sel,

input  [31:0]  in_alu_result,
input  [31:0]  in_dmem_data,

output [4:0]   out_rd_waddr,
output        out_rd_wena,
output [31:0]  out_rd_wdata
);

assign out_rd_waddr = in_rd_waddr;
assign out_rd_wena  = in_rd_wena;
assign out_rd_wdata = ~in_rd_sel ? in_dmem_data :
in_alu_result;

endmodule

```

12. pcreg.v

```

`timescale 1ns / 1ns

module pcreg(
    input          clk,
    input          ena,
    input          rst,
    input          stall,
    input  [31:0]  pc_in,
    output reg [31:0] pc_out
);

always@(posedge clk or posedge rst)
begin
    if(rst)
    begin
        pc_out <= 32'h00400000;
    end
    else if(ena)
    begin
        if(stall)

```

```

        begin
            pc_out <= pc_out;
        end
        else
            begin
                pc_out <= pc_in;
            end
        end
    else
        begin
            pc_out <= 32'bz;
        end
    end
end
endmodule

```

13. mux.v

```

`timescale 1ns / 1ns

module mux4_5(
    input [4:0] d0,
    input [4:0] d1,
    input [4:0] d2,
    input [4:0] d3,
    input [1:0] s,
    output reg [4:0] y
);

always@(*)
begin
    case(s)
        2'b00: y <= d0;
        2'b01: y <= d1;
        2'b10: y <= d2;
        2'b11: y <= d3;
    endcase
end

endmodule

```

```

module mux4_32(
    input [31:0] d0,
    input [31:0] d1,
    input [31:0] d2,
    input [31:0] d3,
    input [1:0] s,
    output reg [31:0] y
);

always@(*)
begin
    case(s)
        2'b00: y <= d0;
        2'b01: y <= d1;
        2'b10: y <= d2;
        2'b11: y <= d3;
    endcase
end

endmodule

```

14. controller.v

```

`timescale 1ns / 1ns

module controller(
    input          in_branch,
    input [31:0]    in_instruction,

    output          out_rs_rena,
    output          out_rt_rena,
    output          out_rd_wena,
    output [4:0]    out_rd_addr,
    output          out_rd_sel,

    output          out_dmem_ena,
    output          out_dmem_wena,
    output [1:0]    out_dmem_type,

```

```

output          out_ext_signed,
output          out_alu_a_sel,
output          out_alu_b_sel,
output [3:0]    out_alu_sel,
output [1:0]    out_pc_sel
);

wire [5:0] op      = in_instruction[31:26];
wire [5:0] func    = in_instruction[5:0];
wire [4:0] rs_addr = in_instruction[25:21];
wire [4:0] rt_addr = in_instruction[20:16];
wire [4:0] rd_addr = in_instruction[15:11];

// R-type
wire Add      = (op == 6'b000000 && func == 6'b100000);
wire Addu     = (op == 6'b000000 && func == 6'b100001);
wire Sub      = (op == 6'b000000 && func == 6'b100010);
wire Subu     = (op == 6'b000000 && func == 6'b100011);
wire And      = (op == 6'b000000 && func == 6'b100100);
wire Or       = (op == 6'b000000 && func == 6'b100101);
wire Xor      = (op == 6'b000000 && func == 6'b100110);
wire Nor      = (op == 6'b000000 && func == 6'b100111);
wire Slt      = (op == 6'b000000 && func == 6'b101010);
wire Sltu     = (op == 6'b000000 && func == 6'b101011);
wire Sll      = (op == 6'b000000 && func == 6'b000000);
wire Srl      = (op == 6'b000000 && func == 6'b000010);
wire Sra      = (op == 6'b000000 && func == 6'b000011);
wire Sllv     = (op == 6'b000000 && func == 6'b000100);
wire Srlv     = (op == 6'b000000 && func == 6'b000110);
wire Srav     = (op == 6'b000000 && func == 6'b000111);
wire Jr       = (op == 6'b000000 && func == 6'b001000);
wire Rtype    = (op == 6'b000000);

// I-type
wire Addi     = (op == 6'b001000);
wire Addiu    = (op == 6'b001001);
wire Andi     = (op == 6'b001100);
wire Ori      = (op == 6'b001101);
wire Xori     = (op == 6'b001110);
wire Lw       = (op == 6'b100011);
wire Sw       = (op == 6'b101011);
wire Beq      = (op == 6'b000100);
wire Bne      = (op == 6'b000101);

```

```

wire Slti    = (op == 6'b001010);
wire Sltiu   = (op == 6'b001011);
wire Lui     = (op == 6'b001111);
wire Itype   = Addi | Addiu | Andi | Ori | Xori | Lw | Sw |
Beq | Bne | Slti | Sltiu | Lui;

// J-type
wire J       = (op == 6'b000010);
wire Jal     = (op == 6'b000011);
wire Jtype   = J | Jal;

assign out_rs_rena = Add | Addu | Sub | Subu | And | Or |
Xor | Nor | Slt | Sltu | Sllv | Srlv | Jr | Addi | Addiu | Andi
| Ori | Xori | Lw | Sw | Beq | Bne | Slti | Sltiu;
assign out_rt_rena = Add | Addu | Sub | Subu | And | Or |
Xor | Nor | Slt | Sltu | Sll | Srl | Sra | Sllv | Srlv | Srav |
Beq | Bne;
assign out_rd_wena = Add | Addu | Sub | Subu | And | Or |
Xor | Nor | Slt | Sltu | Sll | Srl | Sra | Sllv | Srlv | Srav |
Addi | Addiu | Andi | Ori | Xori | Lw | Slti | Sltiu | Lui |
Jal;
assign out_rd_sel  = ~Lw;

assign out_dmem_ena = Sw | Lw;
assign out_dmem_wena = Sw;
assign out_dmem_type = 2'b00;

assign out_ext_signed = Addi | Addiu | Lw | Sw | Slti |
Sltiu;

assign out_alu_a_sel = Sll | Srl | Sra;
assign out_alu_b_sel = Addi | Addiu | Andi | Ori | Xori |
Lw | Sw | Slti | Sltiu | Lui;

assign out_pc_sel[1] = J;
assign out_pc_sel[0] = (Beq | Bne) & in_branch;

assign out_alu_sel[3] = Slt | Sltu | Sll | Srl | Sra | Sllv
| Srlv | Srav | Slti | Sltiu | Lui;
assign out_alu_sel[2] = And | Or | Xor | Nor | Sra | Srav |
Andi | Ori | Xori | Lui;
assign out_alu_sel[1] = Sub | Subu | Xor | Nor | Sll | Srl
| Sllv | Srlv | Xori | Beq | Bne;

```



```

    assign out_alu_sel[0] = Addu | Subu | Or | Nor | Sltu | Srl
    | Srlv | Addiu | Ori | Sltiu | Lui;

    mux4_5 mux_rd_addr_urrt(
        .d0(5'bz),
        .d1(rt_addr),
        .d2(rd_addr),
        .d3(5'bz),
        .s({Rtype, Itype}),
        .y(out_rd_addr)
    );

endmodule

```

15. stall.v

```

`timescale 1ns / 1ns

module stall(
    input          in_clk,
    input          in_rst,

    input  [4:0]   in_rs_addr,
    input  [4:0]   in_rt_addr,
    input          in_rs_rena,
    input          in_rt_rena,

    input          in_ex_wena,
    input          in_mem_wena,
    input  [4:0]   in_ex_waddr,
    input  [4:0]   in_mem_waddr,

    output reg     out_stall
);

    reg stall_ltime;

    always @ (negedge in_clk or posedge in_rst)
    begin
        if(in_rst)

```

```

begin
    out_stall    <= 1'b1;
    stall_ltime <= 1'b0;
end
else if (stall_ltime == 0)
begin
    // 流水线 stall 时间到
    if(out_stall)
    begin
        out_stall <= 1'b0;
    end
    // 正常运行 检测冲突
    else
    begin
        // ex 阶段与 id 阶段读写冲突, stall 2 个周期
        if(in_ex_wena && ((in_rs_rena && (in_ex_waddr ==
in_rs_addr)) || (in_rt_rena && (in_ex_waddr == in_rt_addr))))
        begin
            stall_ltime <= 1'b1;
            out_stall    <= 1'b1;
        end
        // ex 阶段与 mem 阶段读写冲突, stall 1 个周期
        else if(in_mem_wena && ((in_rs_rena &&
(in_mem_waddr == in_rs_addr)) || (in_rt_rena && (in_mem_waddr
== in_rt_addr))))
        begin
            stall_ltime <= 1'b0;
            out_stall    <= 1'b1;
        end
    end
end
else
begin
    stall_ltime = stall_ltime - 1;
end
end
endmodule

```

16. regfile.v

```

`timescale 1ns / 1ns

module regfile(
    input          in_clk,
    input          in_rst,

    input          in_rs_rena,
    input          in_rt_rena,
    input          in_rd_wena,
    input  [4:0]    in_rd_addr,
    input  [4:0]    in_rs_addr,
    input  [4:0]    in_rt_addr,
    input  [31:0]   in_rd_data,

    input  [31:0]   init_floors,
    input  [31:0]   init_resistance,

    output reg [31:0] out_rs_data,
    output reg [31:0] out_rt_data,

    output [31:0]    result_attempt_count,
    output [31:0]    result_broken_count,
    output          result_is_last_broken
);

    assign result_attempt_count    = array_reg[4];
    assign result_broken_count    = array_reg[5];
    assign result_is_last_broken  = array_reg[6][0];

    reg [31:0] array_reg[31:0];

    always @(posedge in_clk or posedge in_rst)
    begin
        if (in_rst)
            begin
                array_reg[0] <= 32'b0;
                array_reg[1] <= 32'b0;
                array_reg[2] <= init_floors;
                array_reg[3] <= init_resistance;
                array_reg[4] <= 32'b0;
                array_reg[5] <= 32'b0;
                array_reg[6] <= 32'b0;
                array_reg[7] <= 32'b0;
            end
    end

```

```

        array_reg[8]  <= 32'b0;
        array_reg[9]  <= 32'b0;
        array_reg[10] <= 32'b0;
        array_reg[11] <= 32'b0;
        array_reg[12] <= 32'b0;
        array_reg[13] <= 32'b0;
        array_reg[14] <= 32'b0;
        array_reg[15] <= 32'b0;
        array_reg[16] <= 32'b0;
        array_reg[17] <= 32'b0;
        array_reg[18] <= 32'b0;
        array_reg[19] <= 32'b0;
        array_reg[20] <= 32'b0;
        array_reg[21] <= 32'b0;
        array_reg[22] <= 32'b0;
        array_reg[23] <= 32'b0;
        array_reg[24] <= 32'b0;
        array_reg[25] <= 32'b0;
        array_reg[26] <= 32'b0;
        array_reg[27] <= 32'b0;
        array_reg[28] <= 32'b0;
        array_reg[29] <= 32'b0;
        array_reg[30] <= 32'b0;
        array_reg[31] <= 32'b0;

    end
    else if(in_rd_wena && in_rd_addr != 0)
    begin
        array_reg[in_rd_addr] <= in_rd_data;
    end
end

always @(negedge in_clk)
begin
    if(in_rst)
    begin
        out_rs_data <= 0;
        out_rt_data <= 0;
    end
    else
    begin
        if(in_rs_rena)
        begin

```

```

        out_rs_data <= ((in_rd_wena && (in_rd_addr ==
in_rs_addr)) ? in_rd_data : array_reg[in_rs_addr]);
    end
    else
    begin
        out_rs_data <= 32'b0;
    end

    if(in_rt_rena)
    begin
        out_rt_data <= ((in_rd_wena && (in_rd_addr ==
in_rt_addr)) ? in_rd_data : array_reg[in_rt_addr]);
    end
    else
    begin
        out_rt_data <= 32'b0;
    end
    end
end
end

endmodule

```

17. alu.v

```

`timescale 1ns / 1ns

module alu(
    input    [31:0]  a,
    input    [31:0]  b,
    output    [31:0]  y,
    input    [3:0]    aluc,
    output          zero,
    output          carry,
    output          negative,
    output          overflow
);

    wire signed [31:0] signedA, signedB;
    reg [32:0] ans;

```

```

assign signedA = a;
assign signedB = b;

parameter ADD    = 4'b0000;
parameter ADDU   = 4'b0001;
parameter SUB    = 4'b0010;
parameter SUBU   = 4'b0011;
parameter AND    = 4'b0100;
parameter OR     = 4'b0101;
parameter XOR    = 4'b0110;
parameter NOR    = 4'b0111;
parameter SLT    = 4'b1000;
parameter SLTU   = 4'b1001;
parameter SLL    = 4'b1010;
parameter SRL    = 4'b1011;
parameter SRA    = 4'b1100;
parameter LUI    = 4'b1101;

always@ (*)
begin
    case(aluc)
        ADD:
            begin
                ans = signedA + signedB;
            end
        ADDU:
            begin
                ans = a + b;
            end
        SUB:
            begin
                ans = signedA - signedB;
            end
        SUBU:
            begin
                ans = a - b;
            end
        AND:
            begin
                ans = a & b;
            end
        OR:
            begin

```

```

        ans = a | b;
    end
XOR:
begin
    ans = a ^ b;
end
NOR:
begin
    ans = ~(a | b);
end
SLT:
begin
    ans = (signedA < signedB);
end
SLTU:
begin
    ans = (a < b);
end
SLL:
begin
    ans = (b << a);
end
SRL:
begin
    if(a == 0)
        { ans[31:0], ans[32] } = { b, 1'b0 };
    else
        { ans[31:0], ans[32] } = b >> (a - 1);
    end
SRA:
begin
    if(a == 0)
        { ans[31:0], ans[32] } = { signedB, 1'b0 };
    else
        { ans[31:0], ans[32] } = signedB >>> (a -
1);

    end
LUI:
begin
    ans = { b[15:0], 16'b0 };
end
endcase
end

```

```

    assign y          = ans[31:0];
    assign zero       = (ans == 32'b0) ? 1'b1 : 1'b0;
    assign carry      = ans[32];
    assign overflow    = ans[32] ^ ans[31];
    assign negative    = ans[31];

endmodule

```

18. dmem.v

```

`timescale 1ns / 1ns

module dmem(
    input          clk,
    input          ena,
    input          wena,
    input  [31:0]  addr,
    input  [1:0]   dmem_type,
    input  [31:0]  data_in,
    output [31:0]  data_out
);

    reg [31:0] ram[0:2047];

    always @(negedge clk)
    begin
        if(ena && wena)
            if(type == 2'b10)
                ram[addr][7:0] <= data_in[7:0];
            else if(type == 2'b01)
                ram[addr][15:0] <= data_in[15:0];
            else if(type == 2'b00)
                ram[addr] <= data_in;
            else
                ram[addr] <= 32'bz;
        else;
    end

    assign data_out = ena ? ram[addr] : 32'bz;

```



```
endmodule
```

19. clk_divider.v

```
`timescale 1ns / 1ns

module clk_divider(
    input      in_clk,
    output reg out_clk = 0
);

    parameter k    = 20;
    integer    cnt = 0;

    always @(posedge in_clk)
    begin
        cnt = (cnt + 1) % (k / 2);
        if(cnt == 0)
            out_clk <= ~out_clk;
    end

endmodule
```

20. seg7x16.v

```
`timescale 1ns / 1ns

module seg7x16(
    input clk,
    input reset,
    input cs,
    input [31:0] i_data,
    output [7:0] o_seg,
    output [7:0] o_sel
);

    reg [14:0] cnt;
```

```

always @ (posedge clk, posedge reset)
    if (reset)
        cnt <= 0;
    else
        cnt <= cnt + 1'b1;

wire seg7_clk = cnt[14];

reg [2:0] seg7_addr;

always @ (posedge seg7_clk, posedge reset)
    if(reset)
        seg7_addr <= 0;
    else
        seg7_addr <= seg7_addr + 1'b1;

reg [7:0] o_sel_r;

always @ (*)
    case(seg7_addr)
        7 : o_sel_r = 8'b01111111;
        6 : o_sel_r = 8'b10111111;
        5 : o_sel_r = 8'b11011111;
        4 : o_sel_r = 8'b11101111;
        3 : o_sel_r = 8'b11110111;
        2 : o_sel_r = 8'b11111011;
        1 : o_sel_r = 8'b11111101;
        0 : o_sel_r = 8'b11111110;
    endcase

reg [31:0] i_data_store;
always @ (posedge clk, posedge reset)
    if(reset)
        i_data_store <= 0;
    else if(cs)
        i_data_store <= i_data;

reg [7:0] seg_data_r;
always @ (*)
    case(seg7_addr)
        0 : seg_data_r = i_data_store[3:0];
        1 : seg_data_r = i_data_store[7:4];
        2 : seg_data_r = i_data_store[11:8];

```

```

        3 : seg_data_r = i_data_store[15:12];
        4 : seg_data_r = i_data_store[19:16];
        5 : seg_data_r = i_data_store[23:20];
        6 : seg_data_r = i_data_store[27:24];
        7 : seg_data_r = i_data_store[31:28];
    endcase

    reg [7:0] o_seg_r;
    always @ (posedge clk, posedge reset)
        if(reset)
            o_seg_r <= 8'hff;
        else
            case(seg_data_r)
                4'h0 : o_seg_r <= 8'hC0;
                4'h1 : o_seg_r <= 8'hF9;
                4'h2 : o_seg_r <= 8'hA4;
                4'h3 : o_seg_r <= 8'hB0;
                4'h4 : o_seg_r <= 8'h99;
                4'h5 : o_seg_r <= 8'h92;
                4'h6 : o_seg_r <= 8'h82;
                4'h7 : o_seg_r <= 8'hF8;
                4'h8 : o_seg_r <= 8'h80;
                4'h9 : o_seg_r <= 8'h90;
                4'hA : o_seg_r <= 8'h88;
                4'hB : o_seg_r <= 8'h83;
                4'hC : o_seg_r <= 8'hC6;
                4'hD : o_seg_r <= 8'hA1;
                4'hE : o_seg_r <= 8'h86;
                4'hF : o_seg_r <= 8'h8E;
            endcase

    assign o_sel = o_sel_r;
    assign o_seg = o_seg_r;

endmodule

```